



GIOCO DELL'OCA

Documentazione Completa

CALOGERO FALCONE & FRANCESCO CATANIA

Prefazione

La presente documentazione illustra in modo dettagliato le scelte progettuali e implementative adottate nello sviluppo del software **Gioco dell'Oca**.

L'applicazione è stata realizzata in linguaggio Java utilizzando l'IDE Eclipse, mentre la progettazione UML è stata effettuata con Astah UML.

Nel documento vengono riportate esclusivamente le versioni finali dei materiali prodotti al termine del processo di progettazione; tutte le versioni intermedie sono invece disponibili nelle rispettive sottocartelle della directory "Documentazione".

Infine, sono descritti approfonditamente anche i test svolti e i pattern utilizzati durante lo sviluppo.

Sommario

IDEAZIONE E ANALISI DEI REQUISITI	2
ANALISI ORIENTATA AGLI OGGETTI.....	26
PROGETTAZIONE	49
TESTING	58
REFACTORING.....	62
TEST DI ACCETTAZIONE	62
DESIGN PATTERN APPLICATI	63

IDEAZIONE E ANALISI DEI REQUISITI

1. INTRODUZIONE

La fase di ideazione rappresenta il punto di partenza del progetto e consente di delinearne una prima visione complessiva. In questo stadio è stata condotta un'analisi preliminare, mentre la definizione più dettagliata dei requisiti è stata sviluppata nelle fasi successive.

Per comprendere al meglio le esigenze del progetto e orientarne correttamente lo sviluppo, in questa fase sono stati analizzati i seguenti documenti:

- Documento di Visione;
- Modello dei Casi d'Uso;
- Glossario.

2. DOCUMENTO DI VISIONE

Questo sistema software mira a portare il classico **Gioco dell'Oca** nel mondo digitale, offrendo un'esperienza coinvolgente e divertente per giocatori di tutte le età.

L'obiettivo primario è quello di preservare l'essenza e il fascino del *gioco tradizionale* offrendo al contempo nuove opportunità e funzionalità che solo il mondo digitale può fornire.

Si vuole consentire agli utenti di *personalizzare le regole del gioco* in base alle proprie preferenze, ad esempio permettendo la modifica del numero di giocatori o l'aggiunta di regole speciali. Un altro tipo di personalizzazione che gli utenti possono effettuare riguarda *gli scenari e gli oggetti di gioco*.

Il sistema incoraggia l'interazione sociale tra gli utenti consentendo loro di sfidarsi e di giocare insieme in modalità *multiplayer*, senza trascurare coloro che preferiscono godersi una partita in tranquillità sfidando la sorte contro un giocatore comandato dal computer in modalità *singleplayer*.

Un altro punto di forza del sistema è quello di poter giocare ovunque e con chiunque: basta avere un dispositivo sul quale poter eseguire il software e il gioco è fatto!

In sintesi, il nostro sistema software si pone l'obiettivo di portare il Gioco dell'Oca nel mondo digitale, risolvendo i problemi legati alla personalizzazione, alla portabilità e all'interazione sociale, preservando l'essenza e il divertimento del gioco tradizionale.

2.1 Posizionamento

2.1.1 Opportunità di business

Il software avrà l'obiettivo di permettere ad utenti provenienti da tutto il mondo di poter entrare in partita e godersi il classico Gioco dell'Oca, consentendo agli stessi di giocare partite sempre diverse grazie al *sistema di personalizzazione delle regole*.

Ciò è corredato dalla comodità intrinseca che offre il digitale, che, tra le altre cose, svincola i giocatori dal bisogno di uno spazio dove poggiare il tavoliere e dalla necessità di dover riordinare tutto ciò che viene usato nel gioco una volta terminata la partita.

2.1.2 Formulazione del problema

Il sistema vuole risolvere alcune tra le problematiche principali che fanno parte del mondo dei giochi da tavolo.

Si vogliono eliminare le barriere di accesso al gioco dell'oca tradizionale, consentendo agli utenti di godere dell'esperienza su piattaforme digitali, risolvendo allo stesso tempo il problema della sovrapposizione geografica: i giocatori potranno partecipare al gioco dell'oca anche se non sono fisicamente nello stesso luogo, grazie alla modalità *multiplayer*.

Il sistema si occupa, inoltre, della *gestione automatica* delle regole, risparmiando ai giocatori la necessità di tenere traccia delle dinamiche di gioco o delle regole stesse.

Verranno offerte una vasta gamma di *opzioni di personalizzazione* per consentire ai giocatori di adattare il gioco alle loro preferenze e ai loro obiettivi specifici.

2.1.3 Formulazione della posizione del prodotto

Il software è rivolto a persone di tutte le età. Le caratteristiche intrinseche del gioco, come la sua semplicità, la sua natura divertente e la sua facilità di comprensione, unite alla comodità e all'alta possibilità di personalizzazione del digitale, lo rendono un passatempo gradevole adatto a grandi e piccini.

2.2 Descrizione delle parti interessate

2.2.1 Obiettivi a livello dell'utente

L'attore principale del sistema software sarà il Giocatore. Quest'ultimo deve avere la possibilità di eseguire alcune azioni fondamentali per l'avanzamento del gioco, quali: lanciare il dado, visualizzare la casella sul quale è attualmente posizionato, spostarsi all'interno del tabellone di gioco, verificare gli effetti della casella sulla quale è finito, parametrizzare il gioco scegliendo delle regole e degli scenari personalizzati e tenere traccia dei movimenti degli avversari.

Un'altra tipologia di utente che può interagire con il sistema è quella dell'Amministratore: quest'ultimo ha il compito di gestire tutto ciò che è parametrizzabile dall'utente, come ad esempio le regole, gli scenari, i dadi, le pedine e i tabelloni. Egli avrebbe il compito di introdurre nuovi tipi di personalizzazione del gioco, in modo da poter ampliare sempre di più la gamma di scelta dei giocatori.

2.3 Descrizione generale del progetto

2.3.1 Punto di vista del prodotto

Il sistema è progettato per essere facilmente utilizzabile da persone di qualsiasi fascia d'età e permetterà agli utenti di potersi godere delle partite al classico Gioco dell'Oca, superando le limitazioni del gioco da tavolo originale. Utenti provenienti da qualunque parte del mondo possono riunirsi in partita e giocare da dove desiderano, cimentandosi in sessioni di gioco sempre diverse e mai noiose grazie al sistema di personalizzazione delle regole e degli scenari che il software mette a disposizione.

2.3.2 Riepilogo delle caratteristiche del sistema

Il sistema presenterà le caratteristiche elencate di seguito:

- Avvio partita contro il computer (modalità singleplayer)
- Avvio partita contro altri giocatori (modalità multiplayer)
- Personalizzazione delle regole
- Personalizzazione degli oggetti di gioco (dadi e pedine)
- Personalizzazione degli scenari
- Gestione degli elementi personalizzabili (aggiunta, modifica o rimozione)

3. MODELLO DEI CASI D'USO

3.1 Requisiti

Il software ha l'obiettivo di dare agli utenti la possibilità di giocare al classico Gioco dell'Oca, permettendo loro di cimentarsi in partite avvincenti e sempre diverse grazie al sistema di personalizzazione. Quest'ultimo viene gestito dalla figura dell'amministratore: egli potrà tenere aggiornato il catalogo dei possibili set di regole e delle personalizzazioni (che includono scenari, dadi e pedine) in modo da offrire agli utenti la facoltà di disputare partite diverse a seconda delle proprie preferenze.

Il sistema deve poter gestire le seguenti attività:

- Avvio di una partita singleplayer.
- Creazione di una partita multiplayer.
- Unione ad una partita multiplayer precedentemente creata.
- Implementazione di tutte le regole del gioco dell'oca, inclusi i movimenti dei giocatori post lancio dei dadi, le caselle speciali e le condizioni di vittoria.
- Gestione dei turni dei giocatori.
- Scelta di regole ad hoc e personalizzazioni da parte del giocatore che crea la partita.
- Creazioni di set di regole predefiniti da parte dell'amministratore.
- Creazione di nuovi scenari e nuove personalizzazioni per dadi e pedine da parte dell'amministratore.

3.2 Obiettivi e casi d'uso

Dall'analisi dei requisiti, sono stati ricavati gli attori principali che interagiranno con il sistema e gli obiettivi di cui quest'ultimo vuole occuparsi.

ATTORE	OBIETTIVO	CASO D'USO
Giocatore	Configurare e avviare una partita singleplayer.	UC1: Configurazione e avvio partita singleplayer
Giocatore	Giocare una partita singleplayer.	UC2: Partita singleplayer
Giocatore (Host)	Creare una partita multiplayer.	UC3: Creazione partita multiplayer
Giocatore (Guest)	Unirsi ad una lobby multiplayer.	UC4: Unione a lobby multiplayer
Giocatore	Giocare una partita multiplayer.	UC5: Partita multiplayer
Giocatore	Gestire le impostazioni di sistema.	UC6: Gestione impostazioni utente
Amministratore	Gestione dei set di regole predefinite selezionabili dai giocatori.	UC7: Gestione set di regole predefinite
Amministratore	Gestire gli scenari di gioco selezionabili dall'utente.	UC8: Gestione scenari di gioco
Amministratore	Gestire gli elementi di gioco personalizzabili selezionabili dall'utente.	UC9: Gestione elementi di gioco personalizzabili

3.3 Casi d'uso

Di seguito verrà fornita una descrizione in formato dettagliato dei casi d'uso individuati.

I casi d'uso *UC1: Configurazione e avvio partita singleplayer* e *UC2: Partita singleplayer* sono stati prodotti durante la fase di ideazione e non sono stati modificati durante le iterazioni, mentre i restanti casi d'uso sono stati definiti dettagliatamente durante l'elaborazione delle iterazioni che li implementano.

3.3.1 Caso d'uso UC1: Configurazione e avvio partita singleplayer

Di seguito viene riportato nel dettaglio il caso d'uso UC1: Configurazione e avvio partita singleplayer. Questo caso d'uso permette al giocatore di configurare una partita singleplayer al Gioco dell'Oca, scegliendo le regole, lo scenario e le personalizzazioni da utilizzare per quella sessione di gioco, e avviare la stessa. Questo caso d'uso non presenta modifiche rispetto al caso d'uso presentato in fase di ideazione.

Caso d'uso	Partita singleplayer
Nome del caso d'uso	UC2: Partita singleplayer
Portata	Applicazione Gioco dell'Oca
Livello	Obiettivo Utente
Attore primario	Giocatore
Parti interessate e interessi	Giocatore → Vuole giocare una partita singleplayer al Gioco dell'Oca
Pre-condizioni	Il giocatore deve aver configurato e avviato una partita singleplayer
Garanzia di successo	Il sistema conclude la partita precedentemente avviata
Scenario principale di successo	1. Il sistema imposta l'utente come primo giocatore di turno. 2. Il sistema controlla lo stato del giocatore di turno. 3. L'utente inizia il proprio turno cliccando sul pulsante "Lancia dadi". 4. Il sistema genera N numeri casuali da 1 a 6. 5. La pedina del giocatore di turno si muove in avanti della somma dei numeri che scaturiscono dal lancio dei dadi. 6. In base alla casella del tabellone sulla quale finisce la pedina del giocatore di turno, viene applicato l'effetto appropriato. 7. Il sistema salva il numero della casella sulla quale finisce la pedina del giocatore di turno.

Estensioni

8. Il sistema controlla che la casella sulla quale è finita la pedina del giocatore di turno sia quella finale.
9. Il sistema conclude la partita, indicando come vincitore il giocatore la cui pedina è finita sulla casella finale.
10. L'utente clicca sul pulsante "Torna al menu principale".
11. Il sistema torna alla pagina del menu principale.
- 1a. In qualsiasi momento, l'utente può cliccare sul pulsante "Torna al menu principale".
 1. Il sistema chiede conferma della scelta, avvisando l'utente che perderà i progressi relativi alla partita in corso.
 2. L'utente decide se abbandonare la partita o meno.
 - 2a. L'utente clicca su "Sì".
 1. Il sistema apre la pagina del menu principale, perdendo traccia dei progressi relativi alla partita in corso.
 - 2b. L'utente clicca su "No".
 1. Il sistema permette di continuare normalmente la partita.
 - 1b. In qualsiasi momento, il sistema subisce un arresto improvviso.
 1. L'utente ripristina il sistema.
 2. Il sistema ripristina lo stato della partita.
 - 2a. Il giocatore di turno è nello stato "Bloccato".
 1. Il sistema controlla chi è il giocatore di turno.
 - 1a. Il giocatore di turno è l'utente.
 1. Il sistema imposta come nuovo giocatore di turno il computer e continua con il passo 4.
 - 1b. Il giocatore di turno è il computer.
 1. Il sistema imposta come nuovo giocatore di turno l'utente e continua con il passo 3.
 - 2b. Il giocatore NON è nello stato "Bloccato".
 1. Il sistema controlla chi è il giocatore di turno.
 - 1a. Il giocatore di turno è l'utente.
 1. Il sistema continua con il passo 3.
 - 1b. Il giocatore di turno è il computer.
 1. Il sistema continua con il passo 4.

6a. La casella sulla quale è finito il giocatore di turno è una casella normale.

1. La casella sulla quale finisce la pedina viene impostata alla casella attuale e il sistema continua con il passo 7.

6b. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Oca".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come "Casella attuale" + "Somma dei numeri ottenuti dal lancio dei dadi".

6c. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Ponte".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come "Casella attuale" * 2.

6d. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Locanda".

1. Il sistema imposta lo stato del giocatore di turno a "Bloccato" per il turno successivo.

6e. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Prigione".

1. Il sistema controlla se sulla stessa casella è presente un altro giocatore.

1a. Sulla stessa casella NON è presente un altro giocatore.

1. Il sistema imposta lo stato del giocatore di turno a "Bloccato" indefinitamente.

1b. Sulla stessa casella è presente un altro giocatore.

1. Il sistema imposta lo stato giocatore che è finito sulla casella a "Bloccato" indefinitamente e imposta lo stato del giocatore che era già presente sulla casella a "Sbloccato".

6f. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Labirinto".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come "Casella attuale" - 3.

6g. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Scheletro".

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

1. La nuova casella sulla quale finisce la pedina del giocatore di turno viene reimpostata alla casella di inizio.

8a. La casella sulla quale è finito il giocatore di turno NON è quella finale.

1. Il sistema controlla chi è il giocatore di turno.

1a. Il giocatore di turno è l'utente.

2. Il sistema imposta come nuovo giocatore di turno il computer e ritorna al passo 4.

1b. Il giocatore di turno è il computer.

2. Il sistema imposta come nuovo giocatore di turno l'utente e ritorna al passo 3.

8b. La casella sulla quale è finito il giocatore di turno è quella finale.

1. Il sistema continua con il passo 9.

Dipende dal numero di partite singleplayer che il giocatore vuole avviare

3.3.2 Caso d'uso UC2: Partita singleplayer

Di seguito viene riportato nel dettaglio il caso d'uso UC2: Partita singleplayer. Questo caso d'uso permette al giocatore di entrare in una partita, precedentemente configurata tramite l'UC1, e giocare contro il computer. Questo caso d'uso non presenta sostanziali modifiche rispetto al caso d'uso presentato in fase di ideazione.

Caso d'uso

	Partita singleplayer
Nome del caso d'uso	UC2: Partita singleplayer
Portata	Applicazione Gioco dell'Oca
Livello	Obiettivo Utente
Attore primario	Giocatore
Parti interessate e interessi	Giocatore → Vuole giocare una partita singleplayer al Gioco dell'Oca
Pre-condizioni	Il giocatore deve aver configurato e avviato una partita singleplayer

Garanzia di successo

Il sistema conclude la partita precedentemente avviata

Scenario principale di successo

1. Il sistema imposta l'utente come primo giocatore di turno.
2. Il sistema controlla lo stato del giocatore di turno.
3. L'utente inizia il proprio turno cliccando sul pulsante "Lancia dadi".
4. Il sistema genera N numeri casuali da 1 a 6.
5. La pedina del giocatore di turno si muove in avanti della somma dei numeri che scaturiscono dal lancio dei dadi.
6. In base alla casella del tabellone sulla quale finisce la pedina del giocatore di turno, viene applicato l'effetto appropriato.
7. Il sistema salva il numero della casella sulla quale finisce la pedina del giocatore di turno.
8. Il sistema controlla che la casella sulla quale è finita la pedina del giocatore di turno sia quella finale.
9. Il sistema conclude la partita, indicando come vincitore il giocatore la cui pedina è finita sulla casella finale.
10. L'utente clicca sul pulsante "Torna al menu principale".
11. Il sistema torna alla pagina del menu principale.

Estensioni

- 1a. In qualsiasi momento, l'utente può cliccare sul pulsante "Torna al menu principale".
 1. Il sistema chiede conferma della scelta, avvisando l'utente che perderà i progressi relativi alla partita in corso.
 2. L'utente decide se abbandonare la partita o meno.
- 2a. L'utente clicca su "Sì".
 1. Il sistema apre la pagina del menu principale, perdendo traccia dei progressi relativi alla partita in corso.
- 2b. L'utente clicca su "No".
 1. Il sistema permette di continuare normalmente la partita.
- 1b. In qualsiasi momento, il sistema subisce un arresto improvviso.
 1. L'utente ripristina il sistema.
 2. Il sistema ripristina lo stato della partita.

2a. Il giocatore di turno è nello stato “Bloccato”.

1. Il sistema controlla chi è il giocatore di turno.

1a. Il giocatore di turno è l'utente.

1. Il sistema imposta come nuovo giocatore di turno il computer e continua con il passo 4.

1b. Il giocatore di turno è il computer.

1. Il sistema imposta come nuovo giocatore di turno l'utente e continua con il passo 3.

2b. Il giocatore NON è nello stato “Bloccato”.

1. Il sistema controlla chi è il giocatore di turno.

1a. Il giocatore di turno è l'utente.

1. Il sistema continua con il passo 3.

1b. Il giocatore di turno è il computer.

1. Il sistema continua con il passo 4.

6a. La casella sulla quale è finito il giocatore di turno è una casella normale.

1. La casella sulla quale finisce la pedina viene impostata alla casella attuale e il sistema continua con il passo 7.

6b. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo “Oca”.

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come “Casella attuale” + “Somma dei numeri ottenuti dal lancio dei dadi”.

6c. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo “Ponte”.

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come “Casella attuale” * 2.

6d. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo “Locanda”.

1. Il sistema imposta lo stato del giocatore di turno a “Bloccato” per il turno successivo.

6e. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo “Prigione”.

1. Il sistema controlla se sulla stessa casella è presente un altro giocatore.

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

1a. Sulla stessa casella NON è presente un altro giocatore.

1. Il sistema imposta lo stato del giocatore di turno a "Bloccato" indefinitamente.

1b. Sulla stessa casella è presente un altro giocatore.

1. Il sistema imposta lo stato giocatore che è finito sulla casella a "Bloccato" indefinitamente e imposta lo stato del giocatore che era già presente sulla casella a "Sbloccato".

6f. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Labirinto".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come "Casella attuale" - 3.

6g. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Scheletro".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno viene reimpostata alla casella di inizio.

8a. La casella sulla quale è finito il giocatore di turno NON è quella finale.

1. Il sistema controlla chi è il giocatore di turno.

1a. Il giocatore di turno è l'utente.

2. Il sistema imposta come nuovo giocatore di turno il computer e ritorna al passo 4.

1b. Il giocatore di turno è il computer.

2. Il sistema imposta come nuovo giocatore di turno l'utente e ritorna al passo 3.

8b. La casella sulla quale è finito il giocatore di turno è quella finale.

1. Il sistema continua con il passo 9.

Dipende dal numero di partite singleplayer che il giocatore vuole avviare

3.3.3 Caso d'uso UC3: Creazione partita multiplayer

Il giocatore, che assume il ruolo di *host*, usa il sistema per creare una *lobby* di gioco alla quale altri utenti possono unirsi. Il sistema permette all'*host* di decidere lo scenario e le regole della sessione di gioco che sta creando e di personalizzare i propri oggetti di gioco.

Caso d'uso	Creazione partita multiplayer
Nome del caso d'uso	UC3: Creazione partita multiplayer
Portata	Applicazione Gioco dell'Oca
Livello	Obiettivo Utente
Attore primario	Giocatore
Parti interessate e interessi	Giocatore → Vuole creare una partita multiplayer
Pre-condizioni	
Garanzia di successo	Il sistema imposta le regole e lo scenario per la partita multiplayer che si sta creando e le personalizzazioni per il giocatore <i>host</i>
Scenario principale di successo	1. Il giocatore vuole avviare una partita multiplayer cliccando il pulsante "Nuova Partita Multiplayer". 2. Il sistema apre la pagina di selezione delle regole. 3. Il giocatore seleziona una delle opzioni per la scelta delle regole. 4. Il giocatore clicca sul pulsante di avanzamento "Seleziona scenario". 5. Il sistema apre la pagina di selezione dello scenario. 6. Il giocatore seleziona lo scenario oppure clicca sul pulsante "Seleziona regole". 7. Il giocatore clicca sul pulsante di avanzamento "Seleziona personalizzazioni". 8. Il sistema apre la pagina di personalizzazione degli oggetti di gioco. 9. Il giocatore sceglie una delle opzioni di personalizzazione che verranno applicate per quella partita oppure clicca sul pulsante "Seleziona scenario". 10. Il giocatore clicca sul pulsante "Configura utenti ospiti" e il sistema passa alla schermata di selezione dei giocatori ospiti, salvando le impostazioni precedentemente scelte.
Estensioni	1a. In qualsiasi momento, il giocatore può cliccare sul pulsante "Torna al menu principale".

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

Il sistema apre la pagina del menu principale, perdendo traccia di tutte le scelte precedentemente effettuate.

3a. Il giocatore seleziona l'opzione che gli permette di scegliere un set di regole predefinite.

Il giocatore prende visione dei set di regole predefinite e seleziona il set di regole predefinite desiderato.

Il sistema tiene traccia della scelta effettuata.

Il sistema continua con il passo 4.

3b. Il giocatore seleziona l'opzione che gli permette di scegliere le regole singolarmente.

Il giocatore prende visione delle regole singole e imposta le regole singole desiderate.

Il sistema tiene traccia della scelta effettuata.

Il sistema continua con il passo 4.

6a. Il giocatore seleziona uno scenario.

Il sistema continua con il passo 7, tenendo traccia della scelta effettuata.

6b. Il giocatore clicca sul pulsante "Seleziona regole".

Il sistema torna al passo 2.

9a. Il giocatore clicca sul pulsante "Scegli dadi".

Il giocatore sceglie la personalizzazione che preferisce.

Il sistema tiene traccia della scelta effettuata.

Il sistema torna al passo 8.

9b. Il giocatore clicca sul pulsante "Scegli pedina".

Il giocatore sceglie la personalizzazione che preferisce.

Il sistema tiene traccia della scelta effettuata.

Il sistema torna al passo 8.

9c. Il giocatore clicca sul pulsante "Seleziona scenario"

Il sistema torna al passo 5.

Dipende dal numero di partite multiplayer che il giocatore vuole avviare

3.3.4 Caso d'uso UC4: Unione a lobby multiplayer

Il giocatore, che assume il ruolo di guest, usa il sistema per unirsi ad una lobby di gioco precedentemente creata. Il sistema permette al guest di personalizzare i propri oggetti di gioco (dadi, pedina), ma gli impone di seguire le regole decise dall'host per quella sessione.

Caso d'uso	Unione a lobby multiplayer
Nome del caso d'uso	UC4: Unione a lobby multiplayer
Portata	Applicazione Gioco dell'Oca
Livello	Obiettivo Utente
Attore primario	Giocatori ospiti
Parti interessate e interessi	Giocatori ospiti → Vogliono unirsi ad una partita multiplayer precedentemente creata
Pre-condizioni	Il giocatore <i>host</i> deve aver avviato la partita multiplayer, impostandone regole e scenario
Garanzia di successo	Il sistema registra i giocatori <i>guest</i> , ognuno con le proprie personalizzazioni, e avvia la partita multiplayer
Scenario principale di successo	1. Il giocatore <i>host</i> decide il numero di giocatori ospiti, cliccando sull'apposito pulsante. 2. Il sistema apre la pagina di personalizzazione degli oggetti di gioco per il giocatore ospite attuale. 3. Il giocatore ospite attuale sceglie una delle opzioni di personalizzazione che verranno applicate per quella partita. 4. Il giocatore ospite attuale clicca sul pulsante per configurare il prossimo giocatore <i>guest</i>, oppure, se è l'ultimo, clicca "Avvia partita".
Estensioni	1a. In qualsiasi momento, il giocatore può cliccare sul pulsante "Torna al menu principale". Il sistema apre la pagina del menu principale, perdendo traccia di tutte le scelte precedentemente effettuate. 3a. Il giocatore clicca sul pulsante "Scegli dadi". Il giocatore sceglie la personalizzazione che preferisce. Il sistema tiene traccia della scelta effettuata. Il sistema torna al passo 2. 3b. Il giocatore clicca sul pulsante "Scegli pedina". Il giocatore sceglie la personalizzazione che preferisce. Il sistema tiene traccia della scelta effettuata. Il sistema torna al passo 2.

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

4a. Il giocatore ospite attuale non è l'ultimo in coda per selezionare le proprie personalizzazioni: il pulsante disponibile è "Configura giocatore 'attuale + 1'", che, una volta premuto, fa sì che il sistema torni al passo 2 rendendo il prossimo giocatore ospite il giocatore ospite attuale.

4b. Il giocatore ospite attuale è l'ultimo in coda per selezionare le proprie personalizzazioni: il pulsante disponibile è "Avvia partita", che, una volta premuto, fa sì che il sistema inizi la sessione di gioco con le configurazioni precedentemente scelte.

Dipende dal numero di partite multiplayer che il giocatore vuole avviare

3.3.5 Caso d'uso UC5: Partita multiplayer

Il giocatore usa il sistema per poter giocare ad una partita multiplayer precedentemente configurata e avviata.

Caso d'uso

	Partita multiplayer
Nome del caso d'uso	UC5: Partita multiplayer
Portata	Applicazione Gioco dell'Oca
Livello	Obiettivo Utente
Attore primario	Giocatori (<i>host</i> e <i>guest</i>)
Parti interessate e interessi	Giocatori (<i>host</i> e <i>guest</i>) → Vogliono giocare ad una partita multiplayer precedentemente configurata
Pre-condizioni	Il giocatore <i>host</i> ha configurato le regole e lo scenario e ha scelto le proprie personalizzazioni per la partita corrente; i giocatori <i>guest</i> hanno scelto le proprie personalizzazioni per la partita corrente.
Garanzia di successo	Il sistema registra i giocatori <i>guest</i> , ognuno con le proprie personalizzazioni, e avvia la partita multiplayer
Scenario principale di successo	- Il sistema imposta il giocatore <i>host</i> come primo giocatore di turno. - Il sistema controlla lo stato del giocatore di turno.

Estensioni

3. Il giocatore di turno inizia il proprio turno cliccando sul pulsante "Lancia dadi".
4. Il sistema genera N numeri casuali da 1 a 6.
5. La pedina del giocatore di turno si muove in avanti della somma dei numeri che scaturiscono dal lancio dei dadi.
6. In base alla casella del tabellone sulla quale finisce la pedina del giocatore di turno, viene applicato l'effetto appropriato.
7. Il sistema salva il numero della casella sulla quale finisce la pedina del giocatore di turno.
8. Il sistema controlla che la casella sulla quale è finita la pedina del giocatore di turno sia quella finale.
9. Il sistema conclude la partita, indicando come vincitore il giocatore la cui pedina è finita sulla casella finale.
10. Il giocatore clicca sul pulsante "Torna al menu principale".
11. Il sistema torna alla pagina del menu principale.
- 1a. In qualsiasi momento, il giocatore può cliccare sul pulsante "Torna al menu principale".
 1. Il sistema chiede conferma della scelta, avvisando il giocatore che perderà i progressi relativi alla partita in corso.
 2. Il giocatore decide se abbandonare la partita o meno.
 - 1.1a. Il giocatore clicca su "Sì".
 1. Il sistema apre la pagina del menu principale, perdendo traccia dei progressi relativi alla partita in corso.
 - 1.1b. Il giocatore clicca su "No".
 1. Il sistema permette di continuare normalmente la partita.
 - 2a. Il giocatore di turno è nello stato "Bloccato".
 1. Il sistema imposta come nuovo giocatore di turno il prossimo giocatore e continua con il passo 3.
 - 2b. Il giocatore NON è nello stato "Bloccato".
 1. Il sistema continua con il passo 3.
 - 6a. La casella sulla quale è finito il giocatore di turno è una casella normale.

1. La casella sulla quale finisce la pedina viene impostata alla casella attuale e il sistema continua con il passo 7.

6b. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Oca".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come "Casella attuale" + "Somma dei numeri ottenuti dal lancio dei dadi".

6c. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Ponte".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come "Casella attuale" * 2.

6d. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Locanda".

1. Il sistema imposta lo stato del giocatore di turno a "Bloccato" per il turno successivo.

6e. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Prigione".

1. Il sistema controlla se sulla stessa casella è presente un altro giocatore.

1a. Sulla stessa casella NON è presente un altro giocatore.

1. Il sistema imposta lo stato del giocatore di turno a "Bloccato" indefinitamente.

1b. Sulla stessa casella è presente un altro giocatore.

1. Il sistema imposta lo stato giocatore che è finito sulla casella a "Bloccato" indefinitamente e imposta lo stato del giocatore che era già presente sulla casella a "Sbloccato".

6f. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Labirinto".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come "Casella attuale" - 3.

6g. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Scheletro".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno viene reimpostata alla casella di inizio.

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

8a. La casella sulla quale è finito il giocatore di turno NON è quella finale.

1. Il sistema imposta come nuovo giocatore di turno il prossimo giocatore e ritorna al passo 3.

8b. La casella sulla quale è finito il giocatore di turno è quella finale.

1. Il sistema continua con il passo 9.

Dipende dal numero di partite multiplayer che il giocatore vuole avviare

3.3.6 Caso d'uso UC6: Gestione impostazioni utente

Il giocatore usa il sistema per poter gestire le impostazioni di gioco. Il sistema permette al giocatore di scegliere il proprio nome utente, di impostare delle personalizzazioni di oggetti di gioco predefinite e di effettuare ulteriori azioni legate alla configurazione dell'esperienza di gioco personale.

Caso d'uso

Gestione impostazioni utente

Nome del caso d'uso

UC6: Gestione impostazioni utente

Portata

Applicazione Gioco dell'Oca

Livello

Obiettivo Utente

Attore primario

Giocatore

Parti interessate e interessi

Giocatore → Vuole configurare le impostazioni

Pre-condizioni

Garanzia di successo

Il sistema salva le impostazioni scelte dall'utente

Scenario principale di successo

1. Il giocatore vuole configurare le impostazioni cliccando il pulsante "Gestione impostazioni".
2. Il sistema apre la pagina di gestione delle impostazioni.
3. Il giocatore seleziona le impostazioni che vuole salvare, compilandole secondo la propria scelta. Può infatti decidere di configurare i nomi dei giocatori, lo scenario, le regole, i dadi e le pedine

Estensioni

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

che verranno preselezionati in fase di creazione della partita.

4. Il giocatore clicca sul pulsante di salvataggio "Salva impostazioni".

5. Il sistema salva le impostazioni precedentemente selezionate dall'utente.

1a. In qualsiasi momento, il giocatore può cliccare sul pulsante "Menu", che permette di tornare al menu principale.

Il sistema apre la pagina del menu principale, perdendo traccia di tutte le scelte precedentemente effettuate.

3.3.7 Caso d'uso UC7: Gestione set di regole predefinite

L'amministratore utilizza il sistema per creare, eliminare o modificare dei set di regole predefinite. Il sistema metterà tali set a disposizione del giocatore in fase di configurazione della partita.

Caso d'uso

Gestione set di regole predefinite

Nome del caso d'uso	UC7: Gestione set di regole predefinite
Portata	Applicazione Gioco dell'Oca
Livello	Obiettivo Utente
Attore primario	Amministratore
Parti interessate e interessi	Amministratore → Vuole gestire i set di regole predefinite
Pre-condizioni	
Garanzia di successo	Il sistema salva i set di regole impostati dall'amministratore
Scenario principale di successo	1. L'amministratore vuole accedere al pannello admin cliccando il pulsante "Pannello admin". 2. Il sistema apre il popup di accesso al pannello admin. 3. L'amministratore inserisce le credenziali di accesso. 4. Il sistema apre il pannello admin.

Estensioni

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

5. L'amministratore accede alla pagina di gestione dei set di regole predefinite cliccando il pulsante "Gestione set di regole".
6. Il sistema apre la pagina di gestione dei set di regole predefinite.
7. L'amministratore può aggiungere ed eliminare i set di regole e ha anche la possibilità di modificare le singole regole di ogni set, impostandone il valore.
8. L'amministratore rende effettive le modifiche effettuate cliccando il pulsante "Salva".

3a. L'accesso al pannello degli amministratori fallisce, e il sistema torna al passo 1.

4a. In qualsiasi momento, l'amministratore può cliccare sul pulsante "Torna al menu", che permette di tornare al menu principale.

7a. In qualsiasi momento, l'amministratore può tornare al pannello degli admin cliccando il pulsante "Indietro". Se dovessero esserci delle modifiche non salvate, il sistema non ne terrà traccia.

3.3.8 Caso d'uso UC8: Gestione scenari di gioco

L'amministratore utilizza il sistema per creare, eliminare o modificare degli scenari di gioco. Il sistema metterà tali scenari a disposizione del giocatore in fase di configurazione della partita.

Caso d'uso

Gestione scenari di gioco

Nome del caso d'uso

UC8: Gestione scenari di gioco

Portata

Applicazione Gioco dell'Oca

Livello

Obiettivo Utente

Attore primario

Amministratore

Parti interessate e interessi

Amministratore → Vuole gestire gli scenari

Pre-condizioni

Garanzia di successo

Il sistema salva gli scenari impostati dall'amministratore

Scenario principale di successo

1. L'amministratore vuole accedere al pannello admin cliccando il pulsante "Pannello admin".
2. Il sistema apre il popup di accesso al pannello admin.
3. L'amministratore inserisce le credenziali di accesso.
4. Il sistema apre il pannello admin.
5. L'amministratore accede alla pagina di gestione degli scenari cliccando il pulsante "Gestione scenari".
6. Il sistema apre la pagina di gestione degli scenari.
7. L'amministratore può aggiungere, modificare ed eliminare gli scenari, impostandone anche le varie descrizioni da mostrare per ogni tipologia di casella.
8. L'amministratore rende effettive le modifiche effettuate cliccando il pulsante "Salva".

3a. L'accesso al pannello degli amministratori fallisce, e il sistema torna al passo 1.

4a. In qualsiasi momento, l'amministratore può cliccare sul pulsante "Torna al menu", che permette di tornare al menu principale.

Estensioni

7a. In qualsiasi momento, l'amministratore può tornare al pannello degli admin cliccando il pulsante "Indietro". Se dovessero esserci delle modifiche non salvate, il sistema non ne terrà traccia.

8a. Nel caso in cui l'amministratore non abbia valorizzato, in una o più righe, codice o descrizione dello scenario, il sistema blocca il salvataggio e torna al passo 7.

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

3.3.9 Caso d'uso UC9: Gestione elementi di gioco personalizzabili

L'amministratore utilizza il sistema per creare, eliminare o modificare degli elementi di gioco personalizzabili, come dadi e pedine. Il sistema metterà tali elementi a disposizione del giocatore in fase di configurazione della partita.

Caso d'uso	Gestione elementi di gioco personalizzabili
Nome del caso d'uso	UC9: Gestione elementi di gioco personalizzabili
Portata	Applicazione Gioco dell'Oca
Livello	Obiettivo Utente
Attore primario	Amministratore
Parti interessate e interessi	Amministratore → Vuole gestire le personalizzazioni
Pre-condizioni	
Garanzia di successo	Il sistema salva le personalizzazioni impostate dall'amministratore
Scenario principale di successo	1. L'amministratore vuole accedere al pannello admin cliccando il pulsante "Pannello admin". 2. Il sistema apre il popup di accesso al pannello admin. 3. L'amministratore inserisce le credenziali di accesso. 4. Il sistema apre il pannello admin. 5. L'amministratore accede alla pagina di gestione delle personalizzazioni cliccando il pulsante "Gestione personalizzazioni". 6. Il sistema apre la pagina di gestione delle personalizzazioni. 7. L'amministratore può aggiungere, modificare ed eliminare i dadi e le pedine, impostandone anche il <i>path</i> contenente l'immagine della singola personalizzazione. 8. L'amministratore rende effettive le modifiche effettuate cliccando il pulsante "Salva".
Estensioni	3a. L'accesso al pannello degli amministratori fallisce, e il sistema torna al passo 1. 4a. In qualsiasi momento, l'amministratore può cliccare sul pulsante "Torna al menu", che permette di tornare al menu principale. 7a. In qualsiasi momento, l'amministratore può tornare al pannello degli admin cliccando il pulsante "Indietro". Se dovessero esserci delle modifiche non salvate, il sistema non ne terrà traccia.

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

8a. Nel caso in cui l'amministratore non abbia valorizzato, in una o più righe, codice o descrizione di una delle personalizzazioni, il sistema blocca il salvataggio e torna al passo 7.

4. GLOSSARIO

Termine	Definizione
Amministratore	Figura che gestisce alcuni aspetti del sistema, quali i set di regole predefiniti, gli scenari e gli elementi di gioco personalizzabili, mettendoli a disposizione del giocatore.
Giocatore	Utente che configura, avvia e gioca le partite.
Host	Categoria di giocatore che, nel contesto delle partite <i>multiplayer</i> , si occupa di creare una <i>lobby</i> di gioco alla quale possono unirsi altri giocatori.
Guest	Categoria di giocatore che, nel contesto delle partite <i>multiplayer</i> , si unisce ad una <i>lobby</i> di gioco precedentemente creata da un <i>host</i> .
Lobby	Stanza virtuale all'interno della quale si possono riunire più utenti, in modo da poter giocare insieme nella stessa partita <i>multiplayer</i> .

ANALISI ORIENTATA AGLI OGGETTI

1. INTRODUZIONE

Seguendo l'approccio iterativo ed evolutivo previsto da *Unified Process*, lo sviluppo del software è stato suddiviso in cinque iterazioni. Questa scelta ha permesso di costruire progressivamente il nucleo dell'architettura, affrontando e risolvendo via via i rischi più significativi. Allo stesso tempo, la definizione dei requisiti è stata gestita in modo graduale, riducendo al minimo le conseguenze di eventuali errori di progettazione o implementazione.

2.1 Iterazione 1

Durante la prima iterazione sono stati analizzati i seguenti requisiti:

- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC1: Configurazione e avvio partita singleplayer*.
- Implementazione di un caso d'uso di *Gioco dell'Oca* necessario per gestire le esigenze di inizializzazione per questa iterazione.

2.2 Iterazione 2

Durante la seconda iterazione sono stati analizzati i seguenti requisiti:

- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC2: Partita singleplayer*.
- Aggiornamento di alcune specifiche introdotte con il caso d'uso *UC1: Configurazione e avvio partita singleplayer* per gestire le esigenze di sviluppo per questa iterazione.

2.3 Iterazione 3

Durante la terza iterazione sono stati analizzati i seguenti requisiti:

- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC3: Creazione partita multiplayer*.
- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC4: Unione a lobby multiplayer*.
- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC5: Partita multiplayer*.

2.4 Iterazione 4

Durante la quarta iterazione sono stati analizzati i seguenti requisiti:

- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC6: Gestione impostazioni utente*.
- Aggiornamento di alcune specifiche introdotte con le precedenti iterazioni, per gestire le esigenze di sviluppo dell'iterazione attuale.
- Refactoring, sia lato documentazione sia lato codice, per migliorare leggibilità, manutenibilità e qualità generale del software.

2.5 Iterazione 5

Durante la quinta iterazione verranno analizzati i seguenti requisiti:

- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC7: Gestione set di regole predefinite*.
- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC8: Gestione scenari di gioco*.
- Implementazione dello scenario principale di successo ed estensioni del caso d'uso *UC9: Gestione elementi di gioco personalizzabili*.
- Arricchimento e perfezionamento dei test unitari dell'applicativo.
- Refactoring della logica di gestione degli effetti delle caselle speciali tramite l'implementazione dei pattern *Singleton* e *Strategy*.
- Affinamento di alcuni dei diagrammi UML introdotti nelle precedenti iterazioni.

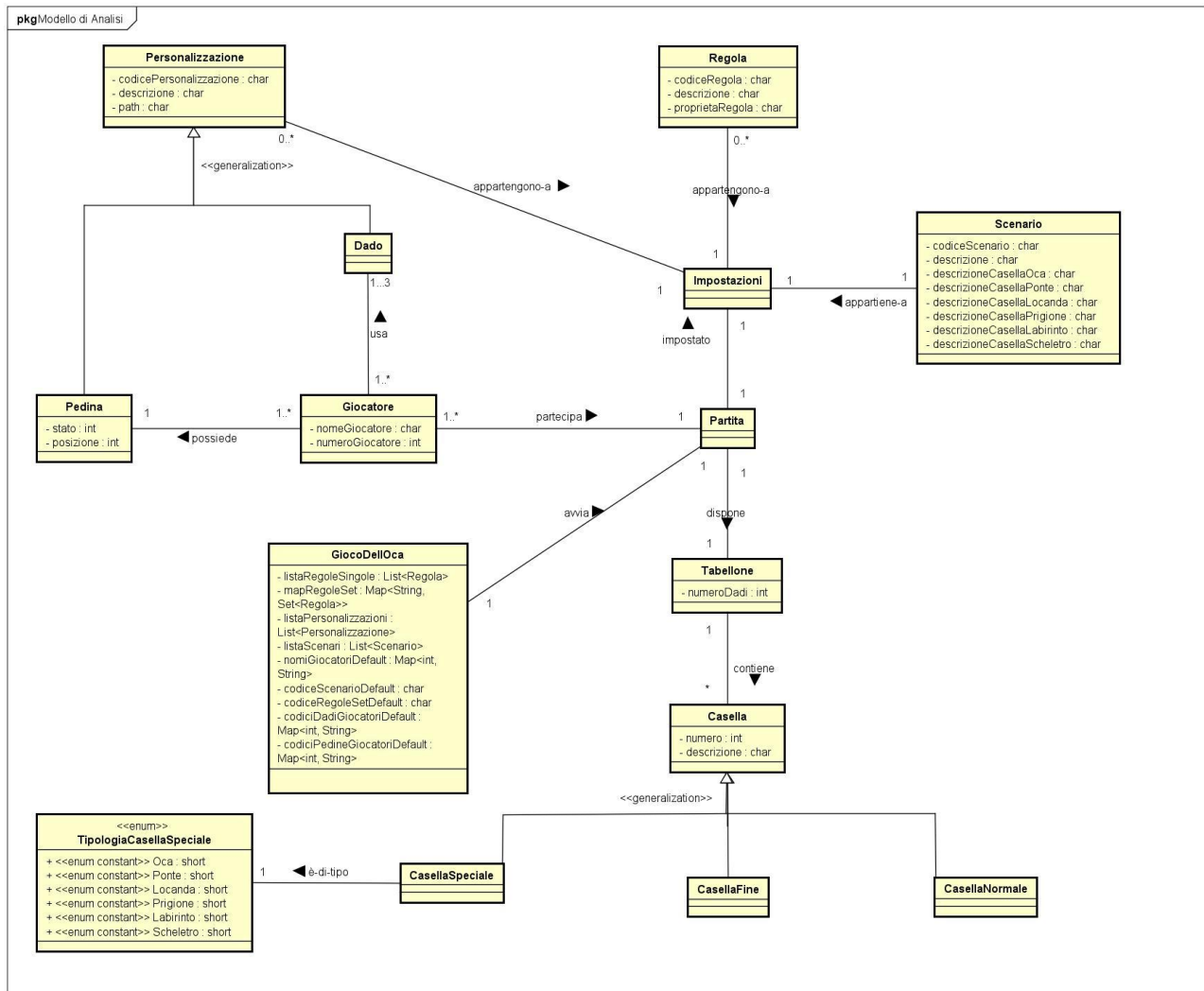
2. MODELLO DI DOMINIO

Bisogna definire il Modello di Dominio, ovvero un elaborato grafico che permette di identificare i concetti, gli attributi e le associazioni più importanti. Dall'analisi dei casi d'uso presi in considerazione, valutando lo scenario principale di successo e le rispettive estensioni, è stato possibile identificare le seguenti classi concettuali:

- **Giocatore**: rappresenta l'attore primario, che interagisce con il sistema per eseguire le operazioni, viene definito nell'Iterazione 1. Nell'iterazione 4 sono stati previsti dei campi aggiuntivi per permettere il salvataggio delle impostazioni utente;
- **GiocoDellOca**: rappresenta il sistema che permette la gestione della partita, viene definito nell'Iterazione 1;
- **Partita**: entità che rappresenta la sessione di gioco e che dispone di impostazioni iniziali per il suo funzionamento viene definito nell'Iterazione 1;
- **Impostazioni**: entità che gestisce regole, scenari e personalizzazioni che verranno usati in partita, viene definito nell'Iterazione 1;
- **Scenario**: lo scenario definisce la narrazione che sottintende ogni partita: inciderà dunque sulla descrizione di ogni casella. Viene scelto uno scenario all'inizio di ogni partita, viene definito nell'Iterazione 1;
- **Regola**: le regole rappresentano la modalità di gioco (rapida, normale o lunga), e dunque il numero di caselle e il numero di dadi che verranno utilizzati dai giocatori nella partita, viene definito nell'Iterazione 1.
- **Personalizzazione**: rappresentano le proprietà di alcuni elementi di gioco sotto il diretto controllo dei giocatori modificabili a piacimento:
 - **Pedina**: rappresenta il Giocatore sulle Caselle del Tabellone. Viene impostato dal Giocatore tramite Impostazioni all'inizio della Partita secondo quanto definito nella Iterazione 1 nel caso di partita singleplayer, e da ciascun giocatore (sia host che guest) secondo quanto definito nell'Iterazione 3 nel caso di partita multiplayer;
 - **Dado**: oggetto che serve per il movimento della Pedina lungo il Tabellone. Viene impostato dal Giocatore tramite Impostazioni all'inizio della Partita secondo quanto definito nella Iterazione 1, e da ciascun giocatore (sia host che guest) secondo quanto definito nell'Iterazione 3 nel caso di partita multiplayer;

- **Tabellone:** si tratta del campo di gioco messo a disposizione dal controller GiocoDellOca tramite Partita. L'interazione con l'oggetto Tabellone non è diretta, ma passa attraverso le Caselle che lo costituiscono;
- **Casella:** rappresentano le varie posizioni che il Giocatore tramite la Pedina può assumere. La Casella può essere di tre tipi: Casella Normale, Casella Speciale e Casella Fine. A sua volta, sono presenti diversi tipi di Casella Speciale: Oca, Ponte, Locanda, Prigione, Labirinto e Scheletro.

Tenendo conto di associazioni e attributi, si costruisce il seguente Modello di Dominio:

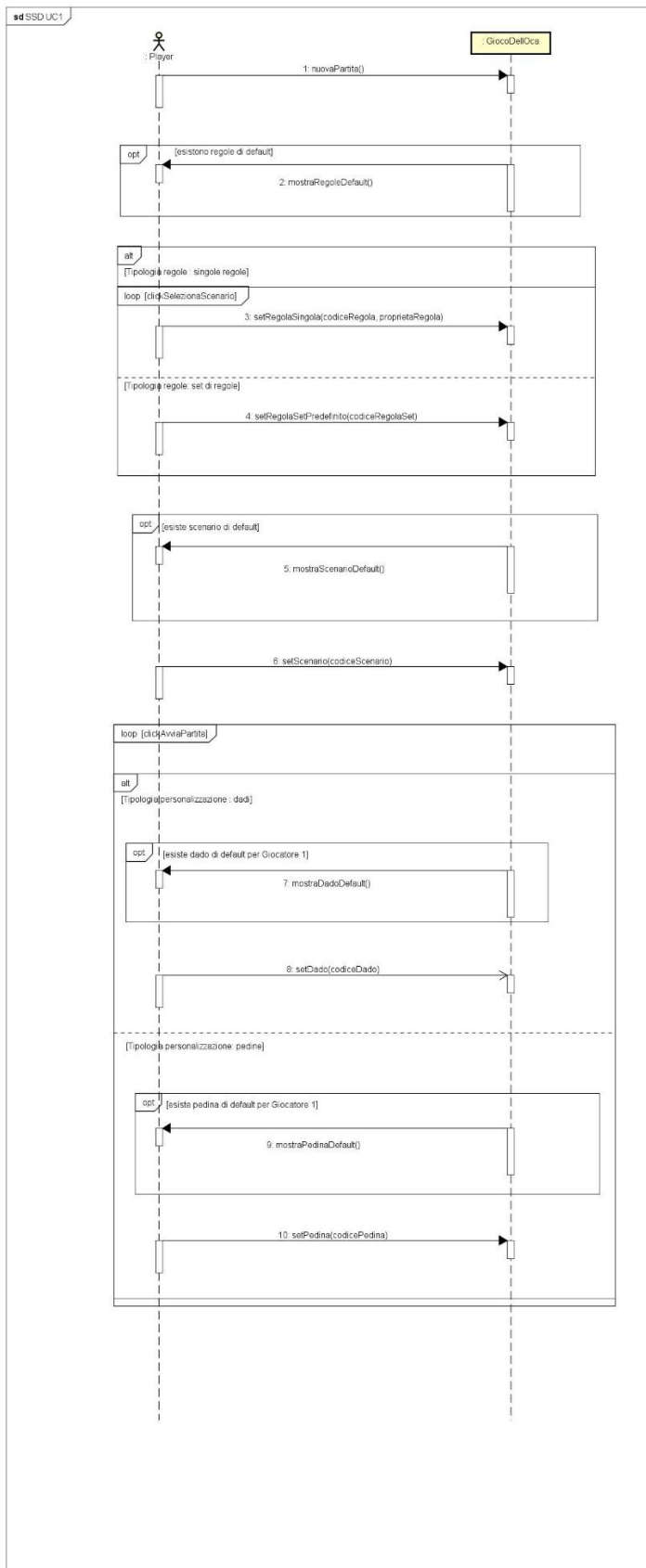


3. SSD E CONTRATTI

Proseguendo con l'Analisi Orientata agli Oggetti, il passo successivo consiste nella realizzazione dei Diagrammi di Sequenza di Sistema (SSD), utili a rappresentare il flusso degli eventi di input e output relativi ai casi d'uso considerati in ciascuna iterazione. Le principali operazioni di sistema individuate all'interno degli SSD saranno successivamente descritte tramite appositi Contratti.

3.1 ITERAZIONE 1

Scenario principale di successo del caso d'uso UC1:



3.1.1 Contratti delle operazioni

Contratto C01 *nuovaPartita*

Operazioni	nuovaPartita();
Riferimenti	Caso d'uso UC1: Configurazione e avvio partita singleplayer;
Pre-condizioni	
Post-condizioni	- È stata creata una nuova istanza <i>partita</i> di tipo <i>Partita</i>; - È stata creata una nuova istanza <i>impostazioni</i> di tipo <i>Impostazioni</i>; - <i>partita</i> è stato associato a <i>GiocoDell'Oca</i> tramite l'associazione <i>avvia</i>.

Contratto C02 *mostraRegoleDefault*

Operazioni	mostraRegoleDefault()
Riferimenti	Caso d'uso UC1: Configurazione e avvio partita singleplayer / Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	- Sono state impostate delle regole di default; - È stata avviata la creazione di una nuova partita.
Post-condizioni	- Sono state preimpostate le eventuali regole di default (precedentemente salvate) nelle apposite selezioni per le regole.

Contratto C03 *setRegolaSingola*

Operazioni	setRegolaSingola (codiceRegola, proprietaRegola)
Riferimenti	Caso d'uso UC1: Configurazione e avvio partita singleplayer;
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i>; - Si è selezionata la <i>tipologiaRegole</i> corrispondente a <i>RegolaSingola</i>;
Post-condizioni	- È stata creata una nuova istanza <i>regola</i> di tipo <i>Regola</i>; - Gli attributi di <i>regola</i> sono stati inizializzati;

Contratto C04 *setRegolaSetPredefinito*

Operazioni	setRegolaSetPredefinito(codiceRegolaSet)
Riferimenti	Caso d'uso UC1: Configurazione e avvio partita singleplayer;
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i>; - Si è selezionata la <i>tipologiaRegole</i> corrispondente a <i>SetDiRegole</i>;
Post-condizioni	- È stata creata una nuova istanza <i>regola</i> di tipo <i>Regola</i>; - Gli attributi di <i>regola</i> sono stati inizializzati;

Contratto C05 *mostraScenarioDefault*

Operazioni	mostraScenarioDefault()
Riferimenti	Caso d'uso UC1: Configurazione e avvio partita singleplayer / Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	- È stato impostato uno scenario di default; - È stata avviata la creazione di una nuova partita.
Post-condizioni	- È stato preimpostato l'eventuale scenario di default (precedentemente salvato) nell'apposita selezione per lo scenario.

Contratto C06 *setScenario*

Operazioni	setScenario (codiceScenario)
Riferimenti	Caso d'uso UC1: Configurazione e avvio partita singleplayer;
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i> ; - È stata creata una nuova istanza <i>regola</i> di tipo <i>Regola</i> ;
Post-condizioni	- È stata creata una nuova istanza <i>scenario</i> di tipo <i>Scenario</i> ; - Gli attributi di <i>scenario</i> sono stati inizializzati;

Contratto C07 *mostraDadoDefault*

Operazioni	mostraDadoDefault()
Riferimenti	Caso d'uso UC1: Configurazione e avvio partita singleplayer / Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	- È stato impostato un dado di default per il giocatore; - È stata avviata la creazione di una nuova partita.
Post-condizioni	- È stato preimpostato l'eventuale dado di default (precedentemente salvato) nell'apposita selezione per il dado del giocatore.

Contratto C08 *setDado*

Operazioni	setDado(codiceDado)
Riferimenti	Caso d'uso UC1: Configurazione e avvio partita singleplayer;
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i> ; - È stata creata una nuova istanza <i>regola</i> di tipo <i>Regola</i> ; - È stata creata una nuova istanza <i>scenario</i> di tipo <i>Scenario</i> ; - Si è scelto di personalizzare il <i>Dado</i> con la <i>tipologiaPersonalizzazione</i> ;
Post-condizioni	- È stata creata una nuova istanza <i>personalizzazioneDado</i> di tipo <i>Personalizzazione</i> ; - Gli attributi di <i>personalizzazioneDado</i> sono stati inizializzati;

Contratto C09 *mostraPedinaDefault*

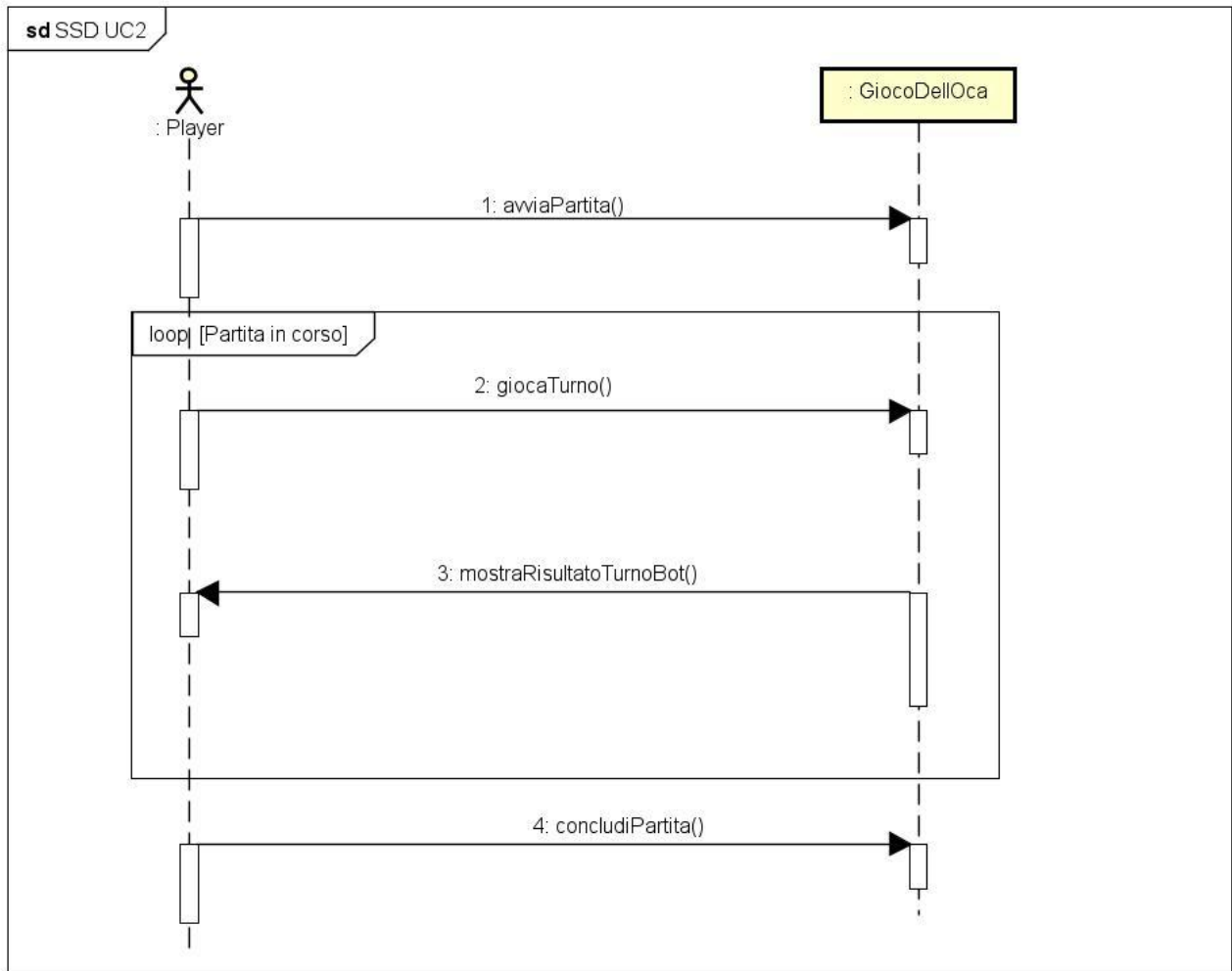
Operazioni	mostraPedinaDefault()
Riferimenti	Caso d'uso UC1: Configurazione e avvio partita singleplayer / Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	- È stata impostata una pedina di default per il giocatore; - È stata avviata la creazione di una nuova partita.
Post-condizioni	- È stato preimpostata l'eventuale pedina di default (precedentemente salvata) nell'apposita selezione per il dado del giocatore.

Contratto C10 *setPedina*

Operazioni	setPedina(codicePedina)
Riferimenti	Caso d'uso UC1: Configurazione e avvio partita singleplayer;
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i> ; - È stata creata una nuova istanza <i>regola</i> di tipo <i>Regola</i> ; - È stata creata una nuova istanza <i>scenario</i> di tipo <i>Scenario</i> ; - Si è scelto di personalizzare la <i>Pedina</i> con la <i>tipologiaPersonalizzazione</i> ;
Post-condizioni	- È stata creata una nuova istanza <i>personalizzazionePedina</i> di tipo <i>Personalizzazione</i> ; - Gli attributi di <i>personalizzazionePedina</i> sono stati inizializzati;

3.2 ITERAZIONE 2

Scenario principale di successo del caso d'uso UC1:



3.2.1 Contratti delle operazioni

Contratto C01 <i>avviaPartita</i>	
Operazioni	avviaPartita();
Riferimenti	Caso d'uso UC2: Partita singleplayer;
Pre-condizioni	- È stata creata una istanza partitaCorrente.
Post-condizioni	- <i>partitaCorrente</i> crea un'istanza di <i>Tabellone</i> e n istanze di <i>Casella</i>, personalizzando il campo da gioco in base alle regole e allo scenario precedentemente selezionati; - <i>partitaCorrente</i> associa al giocatore i dadi e la pedina, in base alle scelte effettuate in fase di configurazione.

Contratto C02 <i>giocaTurno</i>	
Operazioni	giocaTurno();
Riferimenti	Caso d'uso UC2: Partita singleplayer;
Pre-condizioni	- È stata creata una istanza partitaCorrente ed è stata avviata la partita.
Post-condizioni	- Se la pedina associata al giocatore ha uno stato diverso da "Bloccato", il giocatore lancia i dadi e il valore ottenuto dal lancio viene utilizzato per determinare il numero della nuova Casella che verrà associata alla pedina; - Qualora la Casella sia di tipo CasellaSpeciale, viene effettuato un controllo sulla TipologiaCasellaSpeciale: in base a tale proprietà, verranno determinati sia il numero della nuova Casella da associare alla Pedina, sia un eventuale cambiamento di stato della stessa; - Se la Casella è di tipo CasellaFine si procede con l'esecuzione del metodo concludiPartita().

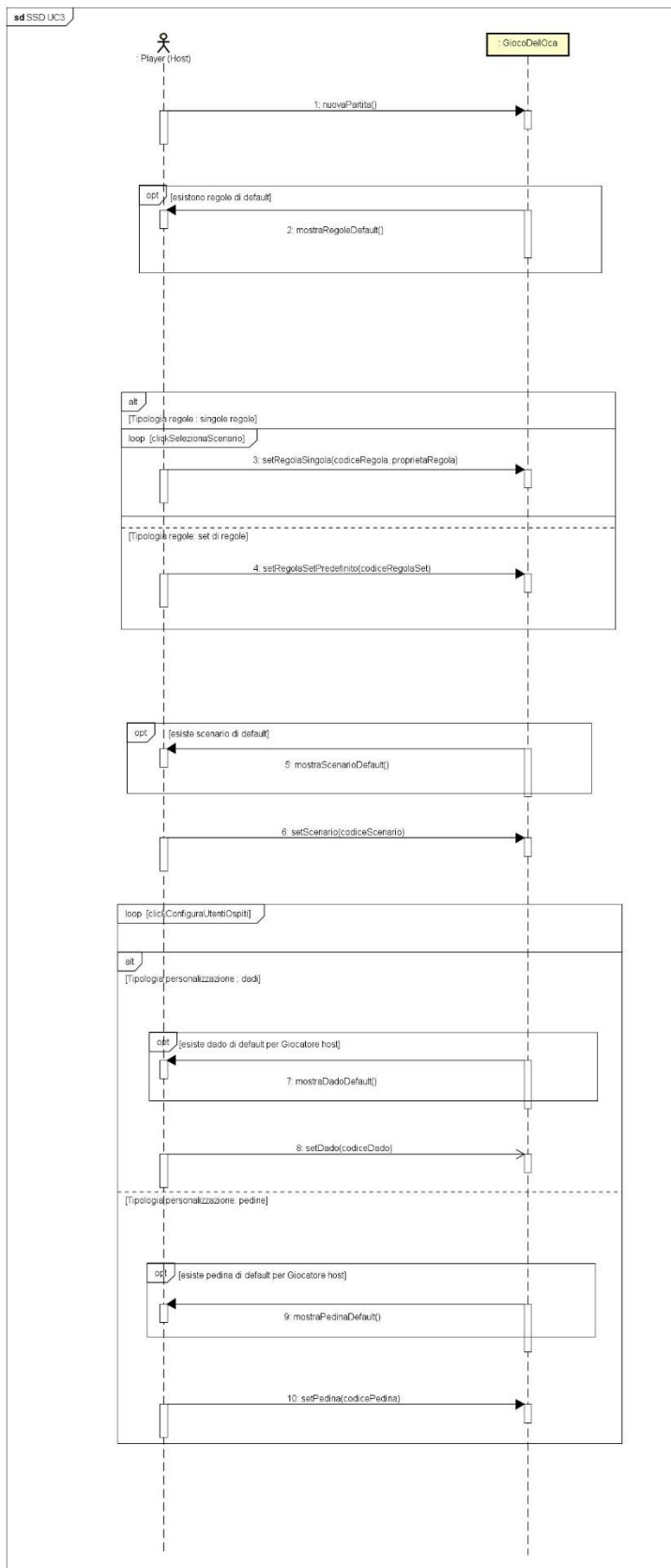
Contratto C03 <i>mostraRisultatoTurnoBot</i>	
Operazioni	mostraRisultatoTurnoBot()
Riferimenti	Caso d'uso UC2: Partita singleplayer;
Pre-condizioni	- Il giocatore ha lanciato i propri dadi e sono state effettuate le valutazioni sulla pedina del giocatore.
Post-condizioni	- Il sistema, in maniera automatica, lancia i dati del bot ed effettua le valutazioni sulla pedina del bot.

Contratto C04 ***concludiPartita***

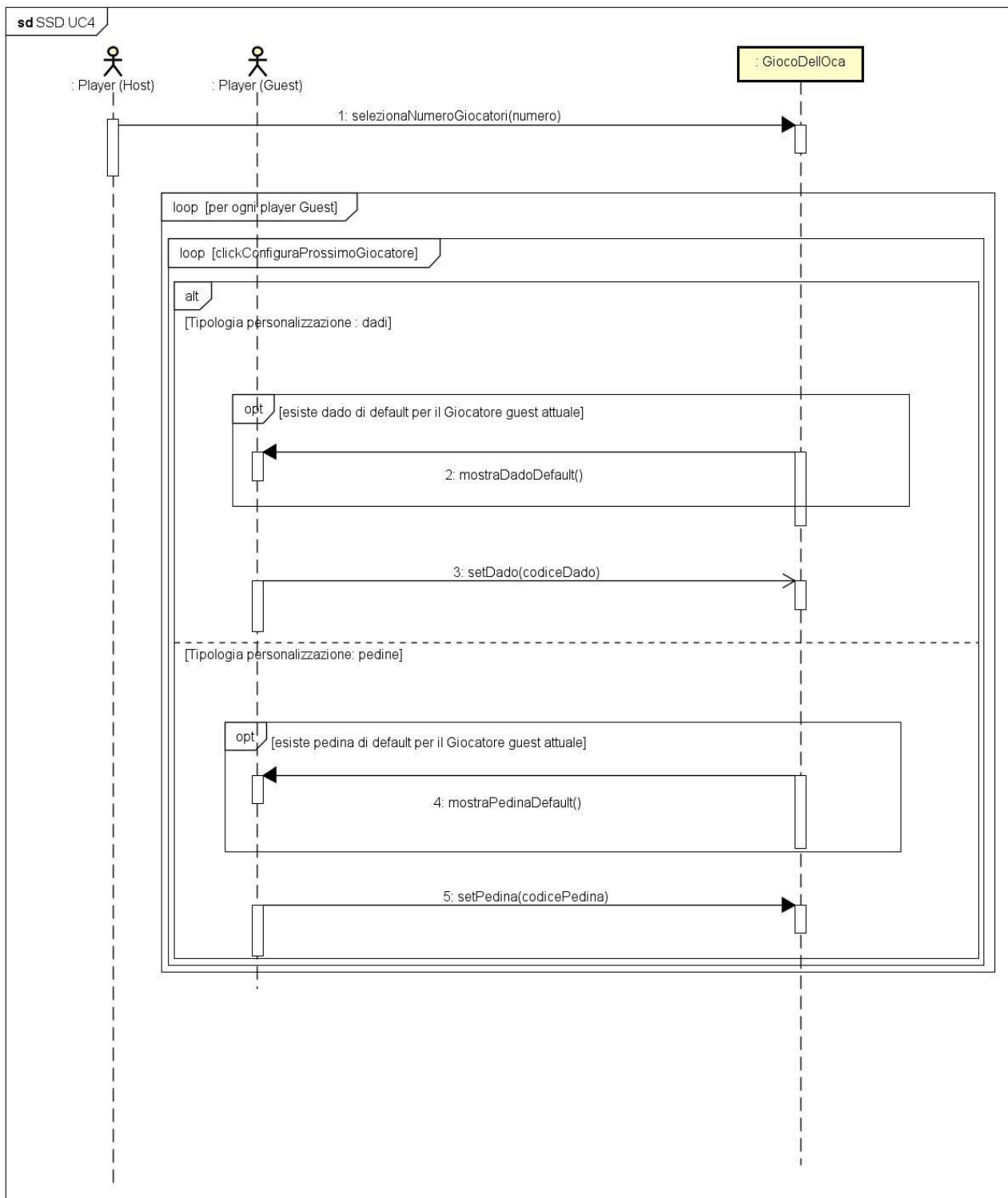
Operazioni	concludiPartita();
Riferimenti	Caso d'uso UC2: Partita singleplayer;
Pre-condizioni	- È stata creata una istanza partitaCorrente ed è stata avviata la partita; - La casella su cui atterra uno dei giocatori è di tipo CasellaFine.
Post-condizioni	- Il gioco finisce e compare a schermo il nome del vincitore con la possibilità di ritornare al menu iniziale.

3.3 ITERAZIONE 3

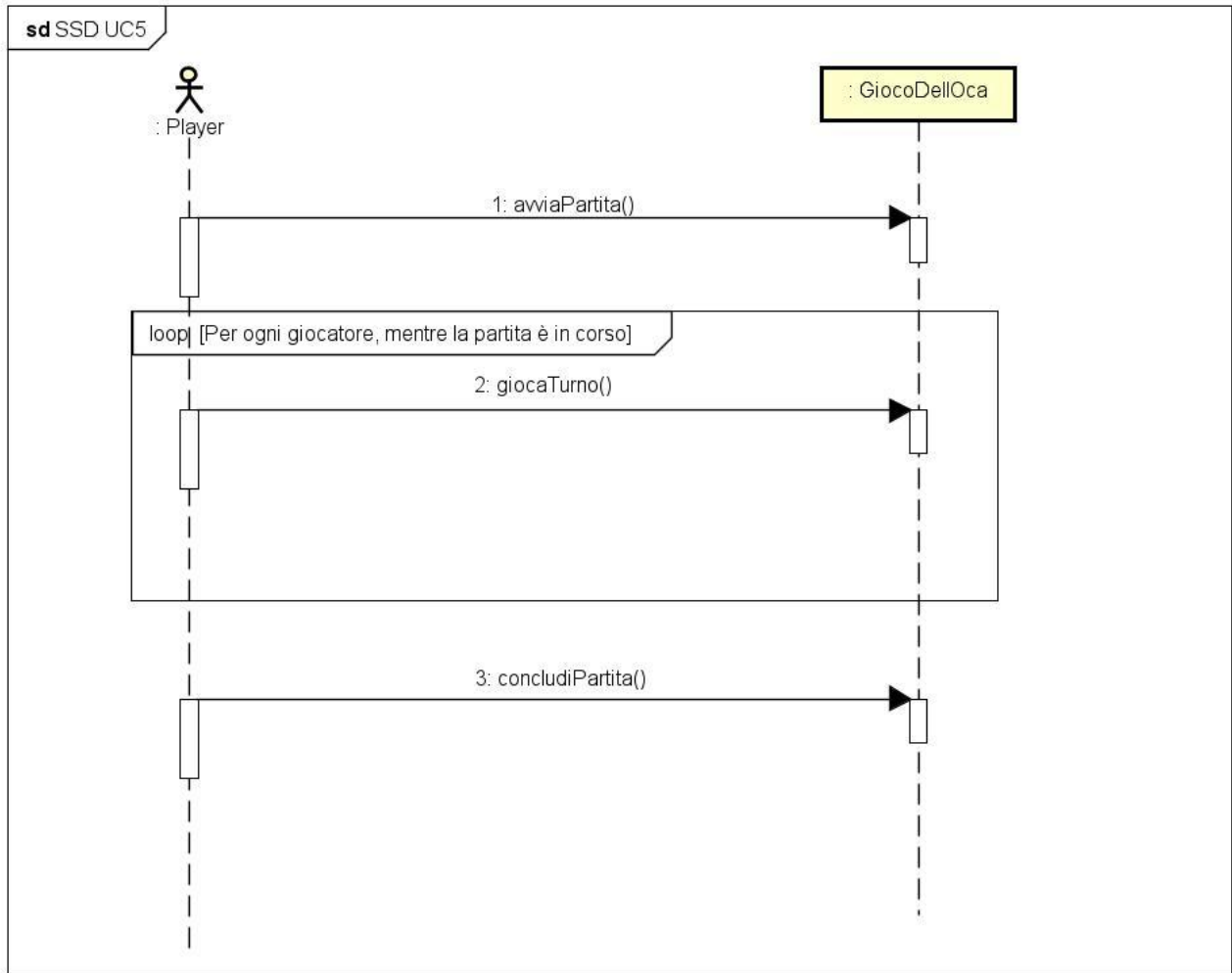
Scenario principale di successo del caso d'uso UC3:



Scenario principale di successo del caso d'uso UC4:



Scenario principale di successo del caso d'uso UC5:



3.3.1 Contratti delle operazioni

Contratto C01 *nuovaPartita*

Operazioni	nuovaPartita()
Riferimenti	Caso d'uso UC3: Creazione partita multiplayer
Pre-condizioni	
Post-condizioni	- È stata creata una nuova istanza <i>partita</i> di tipo <i>Partita</i>; - È stata creata una nuova istanza <i>impostazioni</i> di tipo <i>Impostazioni</i>; - <i>partita</i> è stato associato a <i>GiocoDellOca</i> tramite l'associazione <i>avvia</i>.

Contratto C02 *mostraRegoleDefault*

Operazioni	mostraRegoleDefault()
Riferimenti	Caso d'uso UC3: Configurazione partita multiplayer/ Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	- Sono state impostate delle regole di default; - È stata avviata la creazione di una nuova partita.
Post-condizioni	- Sono state preimpostate le eventuali regole di default (precedentemente salvate) nelle apposite selezioni per le regole.

Contratto C03 *setRegolaSingola*

Operazioni	setRegolaSingola (codiceRegola, proprietaRegola)
Riferimenti	Caso d'uso UC3: Creazione partita multiplayer
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i> ; - Si è selezionata la <i>tipologiaRegole</i> corrispondente a <i>RegolaSingola</i> .
Post-condizioni	- È stata creata una nuova istanza <i>regola</i> di tipo <i>Regola</i> ; - Gli attributi di <i>regola</i> sono stati inizializzati.

Contratto C04 *setRegolaSetPredefinito*

Operazioni	setRegolaSetPredefinito(codiceRegolaSet)
Riferimenti	Caso d'uso UC3: Creazione partita multiplayer
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i> ; - Si è selezionata la <i>tipologiaRegole</i> corrispondente a <i>SetDiRegole</i> .
Post-condizioni	- È stata creata una nuova istanza <i>regola</i> di tipo <i>Regola</i> ; - Gli attributi di <i>regola</i> sono stati inizializzati.

Contratto C05 *mostraScenarioDefault*

Operazioni	mostraScenarioDefault()
Riferimenti	Caso d'uso UC3: Creazione partita multiplayer / Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	- È stato impostato uno scenario di default; - È stata avviata la creazione di una nuova partita.
Post-condizioni	- È stato preimpostato l'eventuale scenario di default (precedentemente salvato) nell'apposita selezione per lo scenario.

Contratto C06 *setScenario*

Operazioni	setScenario (codiceScenario)
Riferimenti	Caso d'uso UC3: Creazione partita multiplayer
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i>; - È stata creata una nuova istanza <i>regola</i> di tipo <i>Regola</i>.
Post-condizioni	- È stata creata una nuova istanza <i>scenario</i> di tipo <i>Scenario</i>; - Gli attributi di <i>scenario</i> sono stati inizializzati.

Contratto C07 *mostraDadoDefault*

Operazioni	mostraDadoDefault()
Riferimenti	Caso d'uso UC3: Creazione partita multiplayer / Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	- È stato impostato un dado di default per il giocatore host; - È stata avviata la creazione di una nuova partita.
Post-condizioni	- È stato preimpostato l'eventuale dado di default (precedentemente salvato) nell'apposita selezione per il dado del giocatore.

Contratto C08 *setDado*

Operazioni	setDado(codiceDado)
Riferimenti	Caso d'uso UC3: Creazione partita multiplayer
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i>; - È stata creata una nuova istanza <i>scenario</i> di tipo <i>Scenario</i>; - Si è scelto di personalizzare il <i>Dado</i> con la <i>tipologiaPersonalizzazione</i>;
Post-condizioni	- È stata creata una nuova istanza <i>personalizzazioneDado</i> di tipo <i>Personalizzazione</i>; - Gli attributi di <i>personalizzazioneDado</i> sono stati inizializzati. (Per giocatore <i>host</i>)

Contratto C09 *mostraPedinaDefault*

Operazioni	mostraPedinaDefault()
Riferimenti	Caso d'uso UC3: Creazione partita multiplayer / Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	- È stata impostata una pedina di default per il giocatore host; - È stata avviata la creazione di una nuova partita.
Post-condizioni	- È stato preimpostata l'eventuale pedina di default (precedentemente salvata) nell'apposita selezione per il dado del giocatore.

Contratto C10 *setPedina*

Operazioni	setPedina(codicePedina)
Riferimenti	Caso d'uso UC3: Creazione partita multiplayer;
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i>; - È stata creata una nuova istanza <i>scenario</i> di tipo <i>Scenario</i>; - Si è scelto di personalizzare la <i>Pedina</i> con la <i>tipologiaPersonalizzazione</i>.
Post-condizioni	- È stata creata una nuova istanza <i>personalizzazionePedina</i> di tipo <i>Personalizzazione</i>; - Gli attributi di <i>personalizzazionePedina</i> sono stati inizializzati. (Per giocatore <i>host</i>)

Contratto C11 *selezionaNumeroGiocatori*

Operazioni	selezionaNumeroGiocatori()
Riferimenti	Caso d'uso UC4: Unione a lobby multiplayer
Pre-condizioni	- È stata creata una istanza <i>partitaCorrente</i> e ne sono stati impostati regole e scenario.
Post-condizioni	- Viene impostato il numero di giocatori che parteciperanno alla partita multiplayer.

Contratto C12 *mostraDadoDefault*

Operazioni	mostraDadoDefault()
Riferimenti	Caso d'uso UC4: Unione a lobby multiplayer / Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	- È stato impostato un dado di default per il giocatore guest attuale; - È stata avviata la creazione di una nuova partita.
Post-condizioni	- È stato preimpostato l'eventuale dado di default (precedentemente salvato) nell'apposita selezione per il dado del giocatore.

Contratto C13 *setDado*

Operazioni	setDado(codiceDado)
Riferimenti	Caso d'uso UC4: Unione a lobby multiplayer
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i>; - È stata creata una nuova istanza <i>scenario</i> di tipo <i>Scenario</i>; - Si è scelto di personalizzare il <i>Dado</i> con la <i>tipologiaPersonalizzazione</i>;
Post-condizioni	- È stata creata una nuova istanza <i>personalizzazioneDado</i> di tipo <i>Personalizzazione</i>; - Gli attributi di <i>personalizzazioneDado</i> sono stati inizializzati. (Per giocatore <i>guest</i>)

Contratto C14 *mostraPedinaDefault*

Operazioni	mostraPedinaDefault()
Riferimenti	Caso d'uso UC4: Unione a lobby multiplayer / Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	- È stata impostata una pedina di default per il giocatore guest attuale; - È stata avviata la creazione di una nuova partita.
Post-condizioni	- È stato preimpostata l'eventuale pedina di default (precedentemente salvata) nell'apposita selezione per il dado del giocatore.

Contratto C15 *setPedina*

Operazioni	setPedina(codicePedina)
Riferimenti	Caso d'uso UC4: Unione a lobby multiplayer
Pre-condizioni	- È stata creata una istanza <i>partita</i> associata a <i>GiocoDell'Oca</i> ; - È stata creata una nuova istanza <i>scenario</i> di tipo <i>Scenario</i> ; - Si è scelto di personalizzare la <i>Pedina</i> con la <i>tipologiaPersonalizzazione</i> .
Post-condizioni	- È stata creata una nuova istanza <i>personalizzazionePedina</i> di tipo <i>Personalizzazione</i> ; - Gli attributi di <i>personalizzazionePedina</i> sono stati inizializzati. (Per giocatore guest)

Contratto C16 *avviaPartita*

Operazioni	avviaPartita()
Riferimenti	Caso d'uso UC5: Partita multiplayer
Pre-condizioni	È stata creata una istanza <i>partitaCorrente</i> ed è stata avviata la partita secondo il Caso d'uso UC4.
Post-condizioni	- <i>partitaCorrente</i> crea un'istanza di <i>Tabellone</i> e n istanze di <i>Casella</i> , personalizzando il campo da gioco in base alle regole e allo scenario precedentemente selezionati; - <i>partitaCorrente</i> associa a tutti i giocatori i dadi e la pedina, in base alle scelte effettuate in fase di configurazione.

Contratto C17 ***giocaTurno***

Operazioni	giocaTurno()
Riferimenti	Caso d'uso UC5: Partita multiplayer
Pre-condizioni	È stata creata una istanza partitaCorrente ed è stata avviata la partita secondo il Caso d'uso UC4.
Post-condizioni	- Se la pedina associata al giocatore ha uno stato diverso da "Bloccato", il giocatore lancia i dadi e il valore ottenuto dal lancio viene utilizzato per determinare il numero della nuova Casella che verrà associata alla pedina; - Qualora la Casella sia di tipo CasellaSpeciale, viene effettuato un controllo sulla TipologiaCasellaSpeciale: in base a tale proprietà, verranno determinati sia il numero della nuova Casella da associare alla Pedina, sia un eventuale cambiamento di stato della stessa. - Se la Casella è di tipo CasellaFine si procede con l'esecuzione del metodo concludiPartita().

Contratto C18 ***concludiPartita***

Operazioni	concludiPartita();
Riferimenti	Caso d'uso UC5: Partita multiplayer
Pre-condizioni	- È stata creata una istanza partitaCorrente ed è stata avviata la partita; - La casella su cui atterra uno dei giocatori è di tipo CasellaFine.
Post-condizioni	- Il gioco finisce e compare a schermo il nome del vincitore con la possibilità di ritornare al menu iniziale.

3.4 ITERAZIONE 4

Scenario principale di successo del caso d'uso UC6:

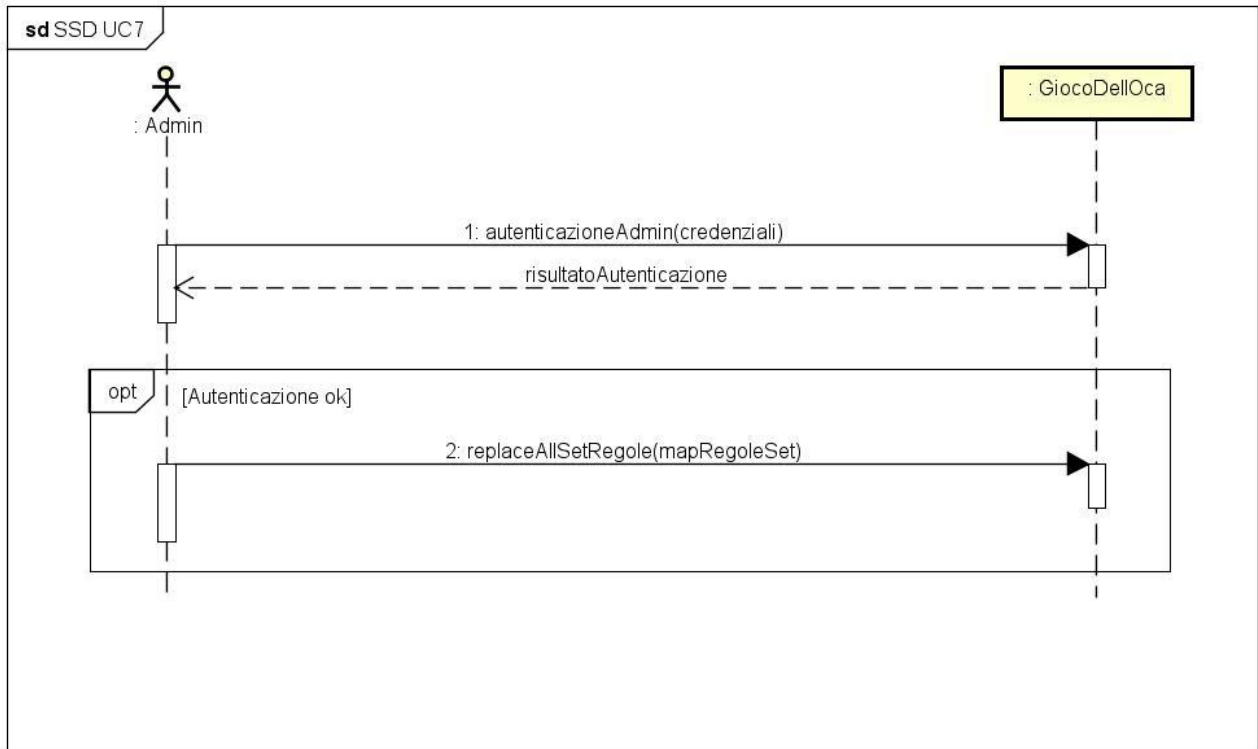


3.4.1 Contratti delle operazioni

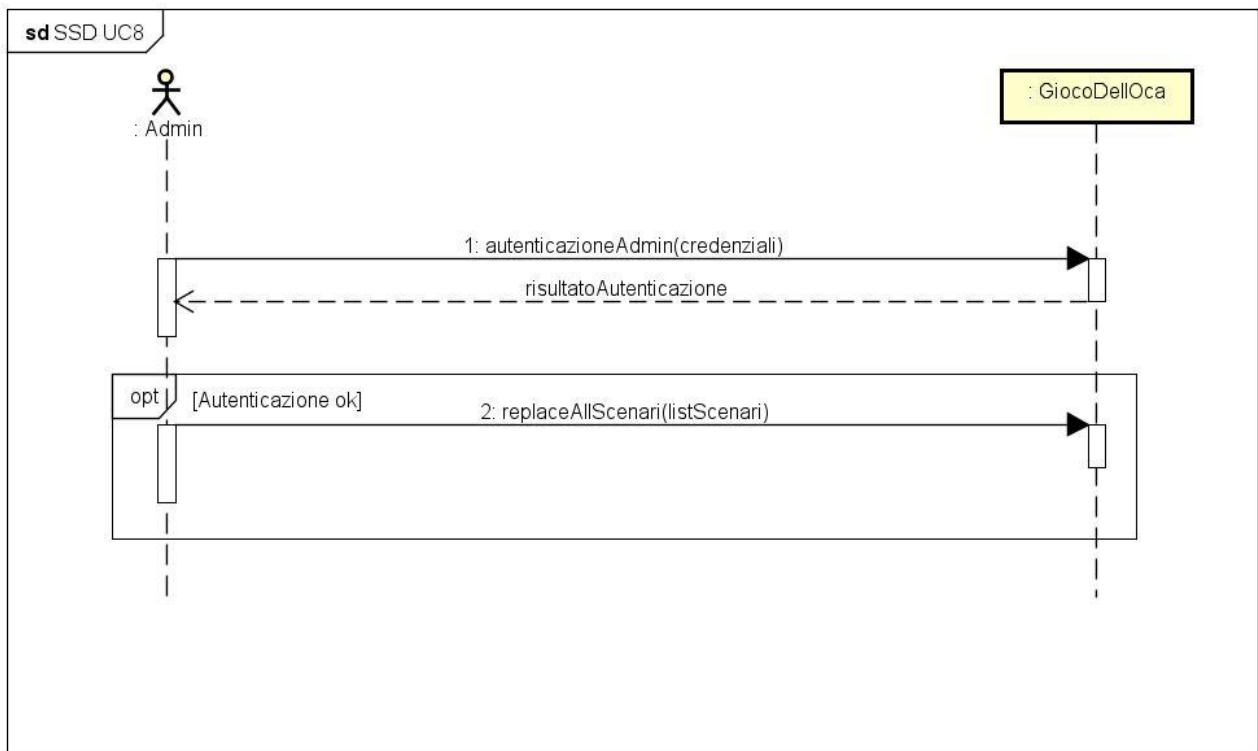
Contratto C01	<i>salvaImpostazioni</i>
Operazioni	salvaImpostazioni(nomiGiocatori, codRegSet, codScenario, dadiGiocatori, pedineGiocatori)
Riferimenti	Caso d'uso UC6: Gestione impostazioni utente
Pre-condizioni	L'utente ha selezionato delle impostazioni da salvare
Post-condizioni	Sono state salvate in <i>GiocoDellOca</i> le impostazioni selezionate dall'utente

3.5 ITERAZIONE 5

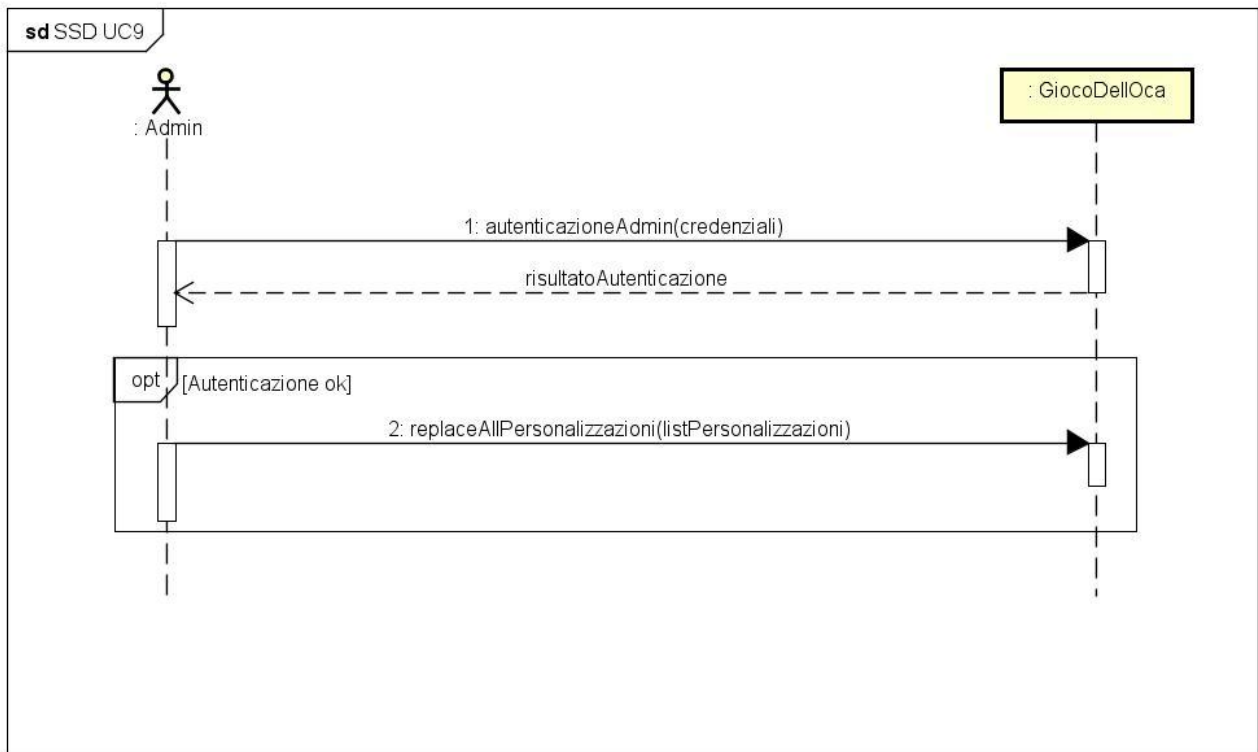
Scenario principale di successo del caso d'uso UC7:



Scenario principale di successo del caso d'uso UC8:



Scenario principale di successo del caso d'uso UC9:



3.4.1 Contratti delle operazioni

Contratto C01 *autenticazioneAdmin*

Operazioni	autenticazioneAdmin(credenziali)
Riferimenti	Casi d'uso UC7, UC8 e UC9
Pre-condizioni	
Post-condizioni	L'amministratore viene autenticato

Contratto C02 *replaceAllSetRegole*

Operazioni	replaceAllSetRegole(mapRegoleSet)
Riferimenti	Caso d'uso UC7: Gestione set di regole predefinite
Pre-condizioni	L'amministratore è stato autenticato con successo
Post-condizioni	Sono stati salvati nel sistema i nuovi set di regole precedentemente impostati

Contratto C03 *replaceAllScenari*

Operazioni	replaceAllScenari(listScenari)
Riferimenti	Caso d'uso UC8: Gestione scenari di gioco
Pre-condizioni	L'amministratore è stato autenticato con successo
Post-condizioni	Sono stati salvati nel sistema i nuovi scenari precedentemente impostati

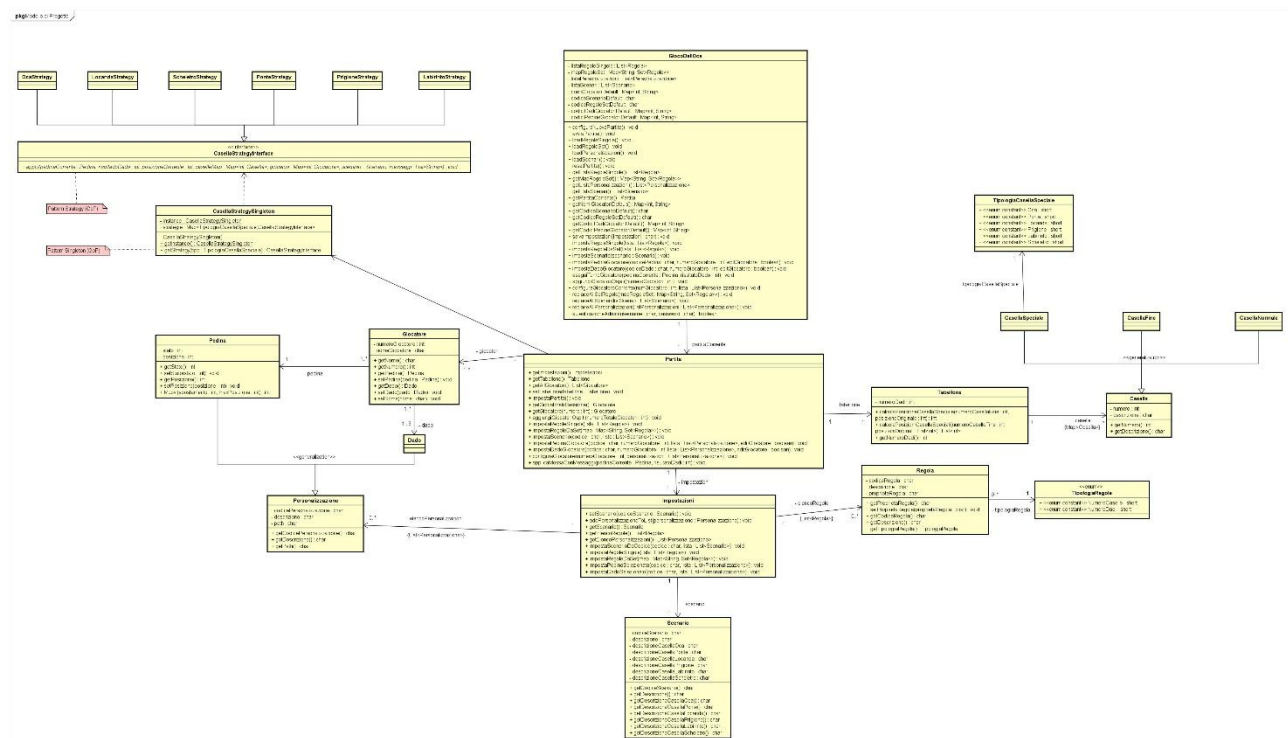
Contratto C04 *replaceAllPersonalizzazioni*

Operazioni	replaceAllPersonalizzazioni(listPersonalizzazioni)
Riferimenti	Caso d'uso UC9: Gestione elementi di gioco personalizzabili
Pre-condizioni	L'amministratore è stato autenticato con successo
Post-condizioni	Sono stati salvati nel sistema le nuove personalizzazioni (dadi e pedine) precedentemente impostate

PROGETTAZIONE

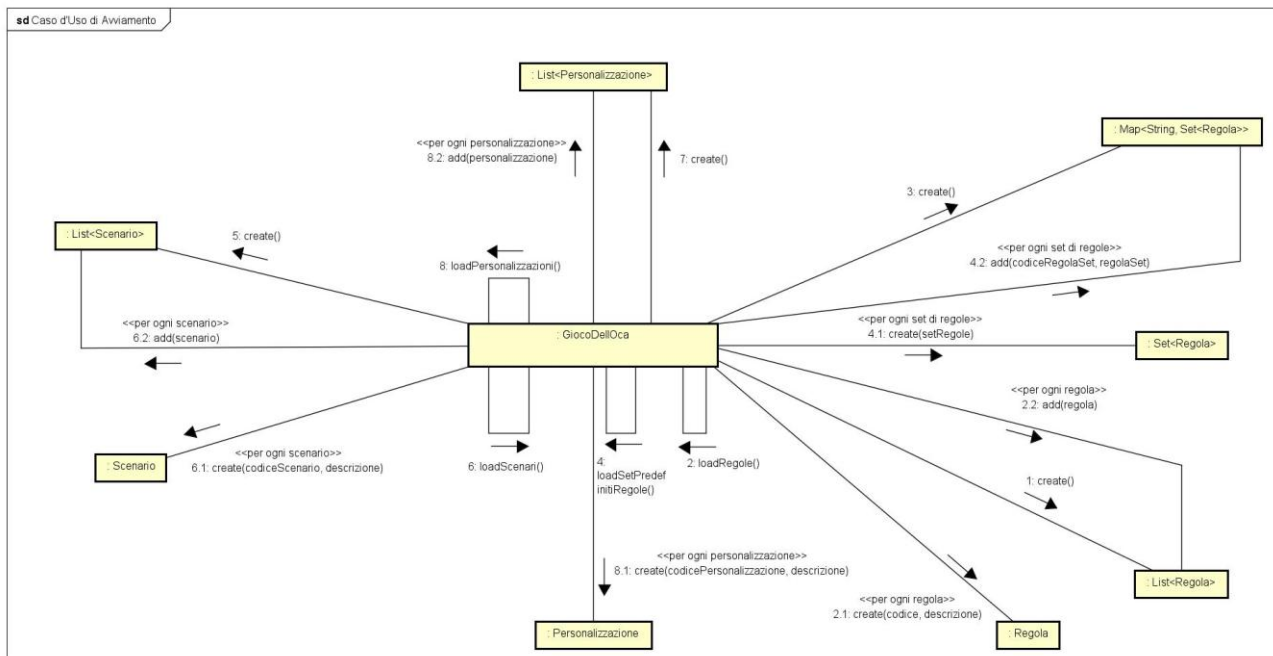
1. DIAGRAMMA DELLE CLASSI

La progettazione orientata agli oggetti, nell'ambito dell'Unified Process, riguarda la definizione degli oggetti software, delle loro responsabilità e delle modalità con cui collaborano per soddisfare i requisiti precedentemente individuati. L'elaborato principale considerato in questa fase è il Modello di Progetto, ossia l'insieme dei diagrammi che descrivono la progettazione logica sia dal punto di vista dinamico, tramite i Diagrammi di Interazione, sia dal punto di vista statico, mediante il Diagramma delle Classi. Tale diagramma è presente all'interno del file "Iterazione5.asta" ed è riportato di seguito:



2. CASO D'USO D'AVVIAMENTO

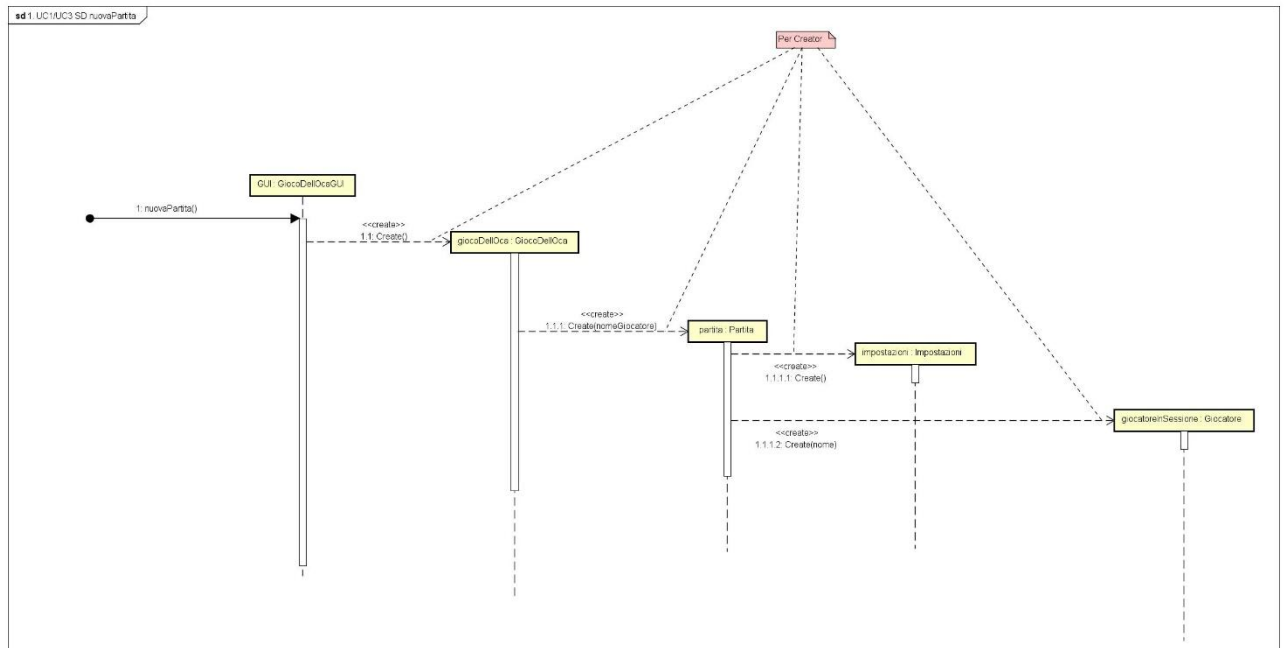
Il caso d'uso di avviamento descrive le operazioni preliminari necessarie per inizializzare il sistema. Viene riportato di seguito:



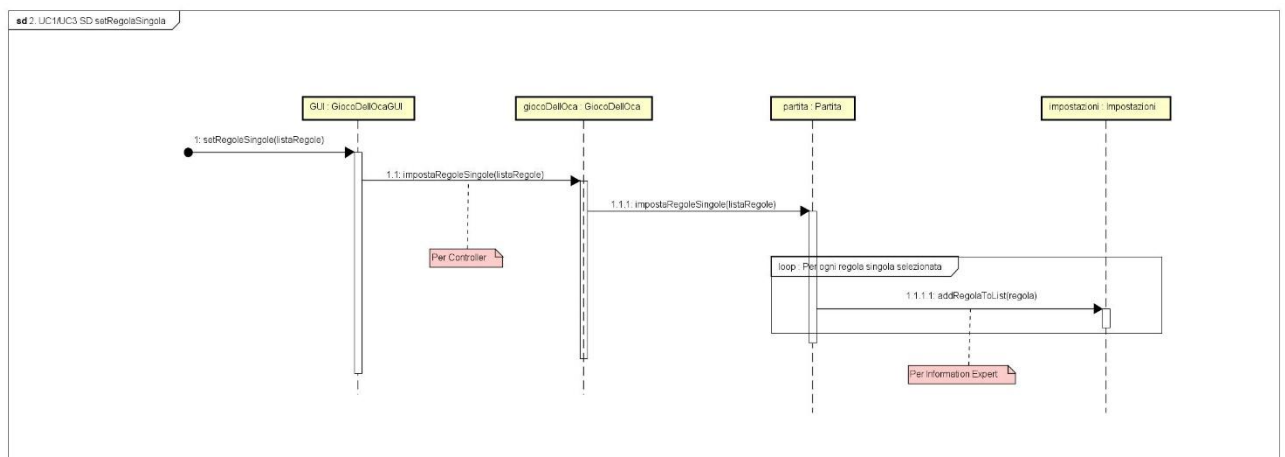
2. CASO D'USO D'AVVIAMENTO

Di seguito sono riportati i diagrammi di sequenza delle operazioni eseguite dal software. Come per il diagramma delle classi, è possibile visualizzarli tutti all'interno del file "Iterazione5.asta".

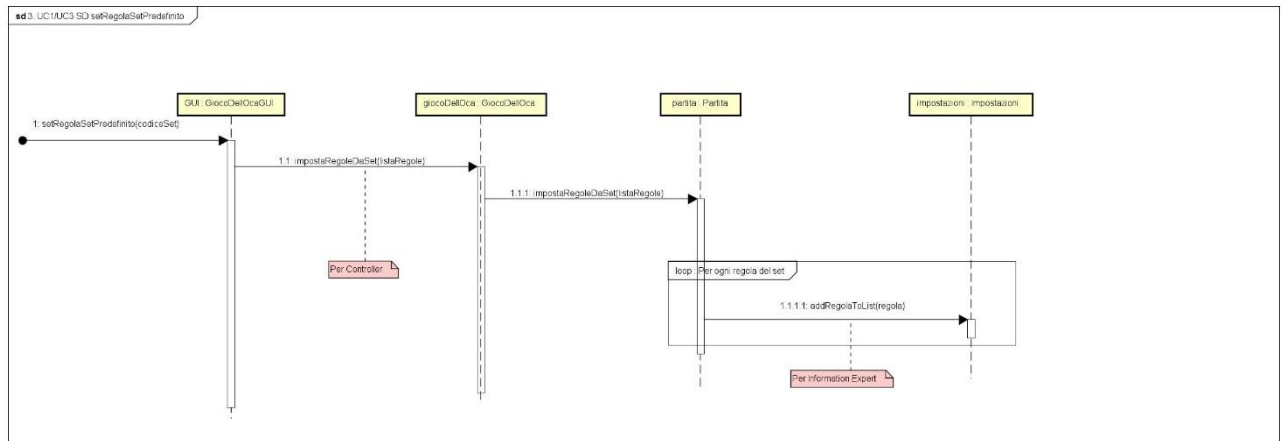
- Nuova partita (UC1/UC3)



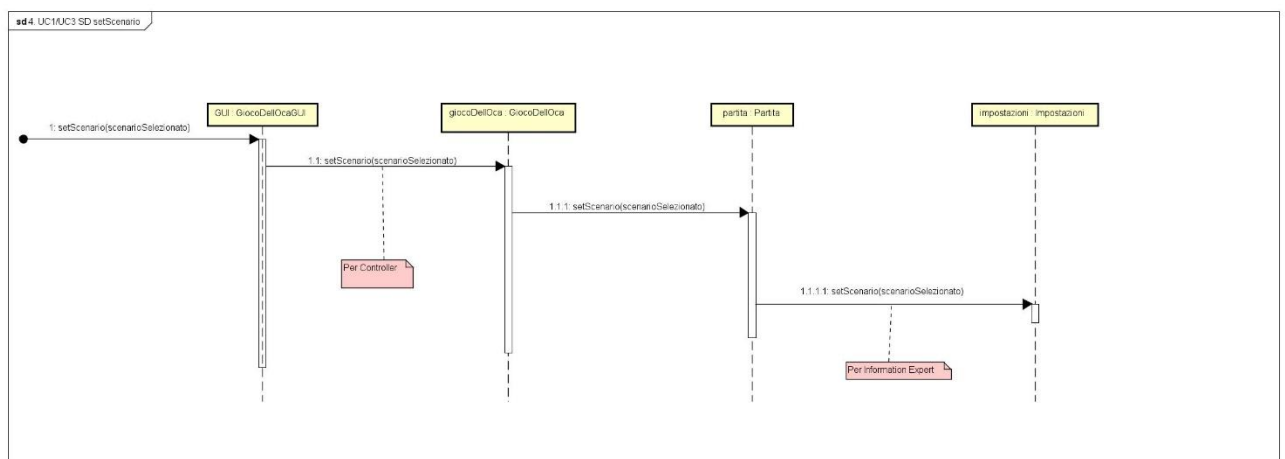
- Set regola singola (UC1/UC3)



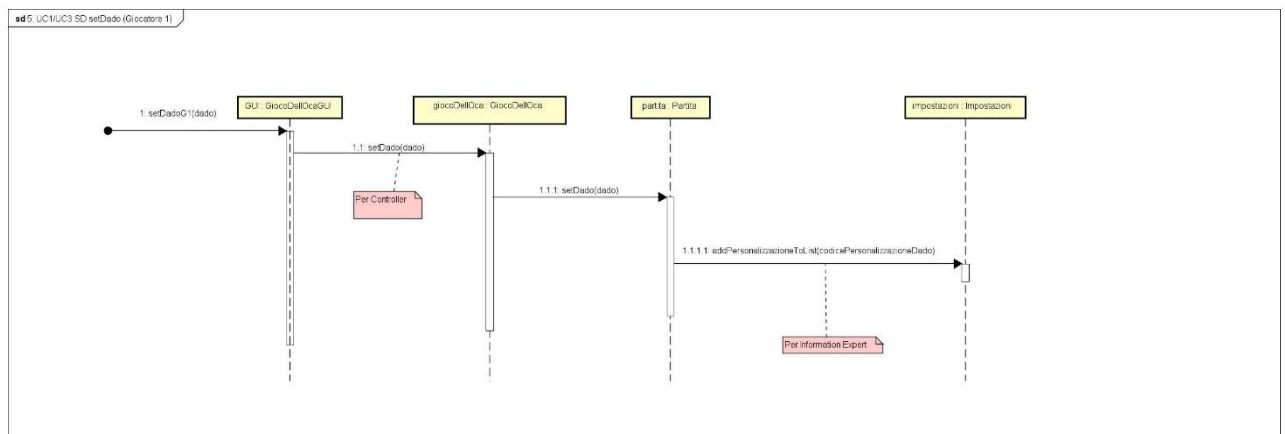
- Set regola set predefinito (UC1/UC3)



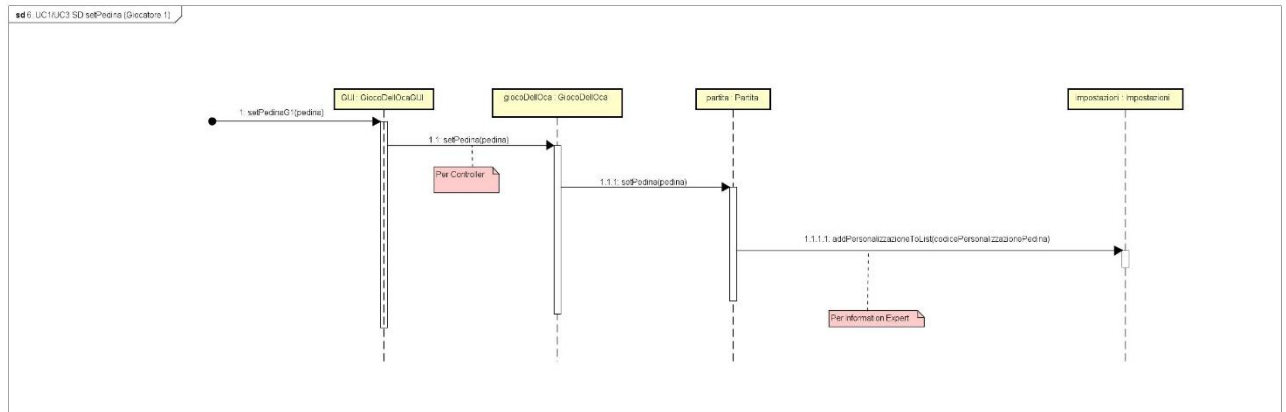
- Set scenario (UC1/UC3)



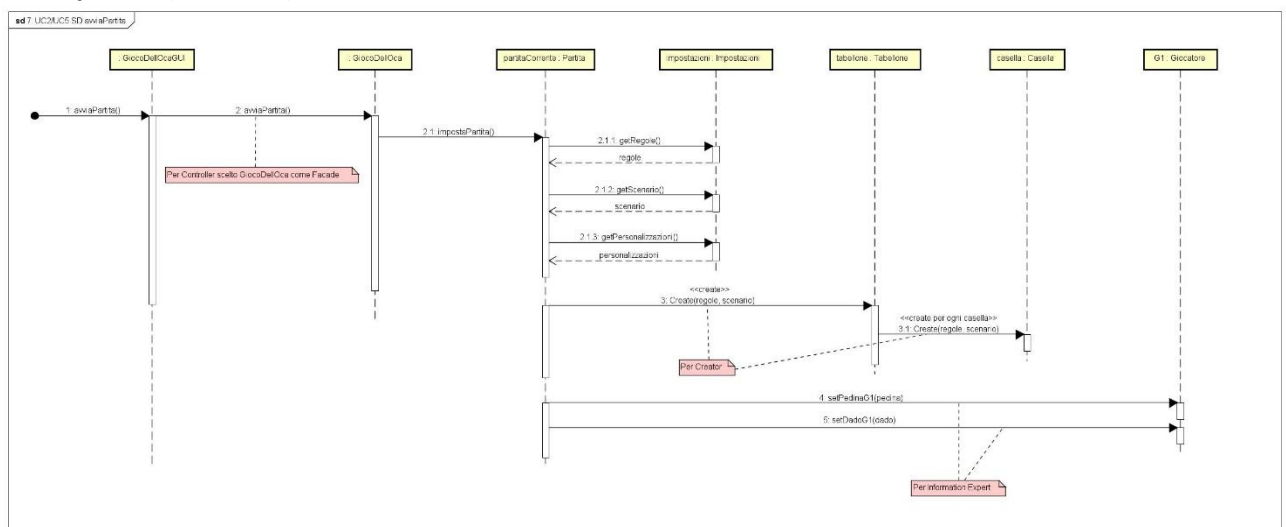
- Set dado (Giocatore 1) (UC1/UC3)



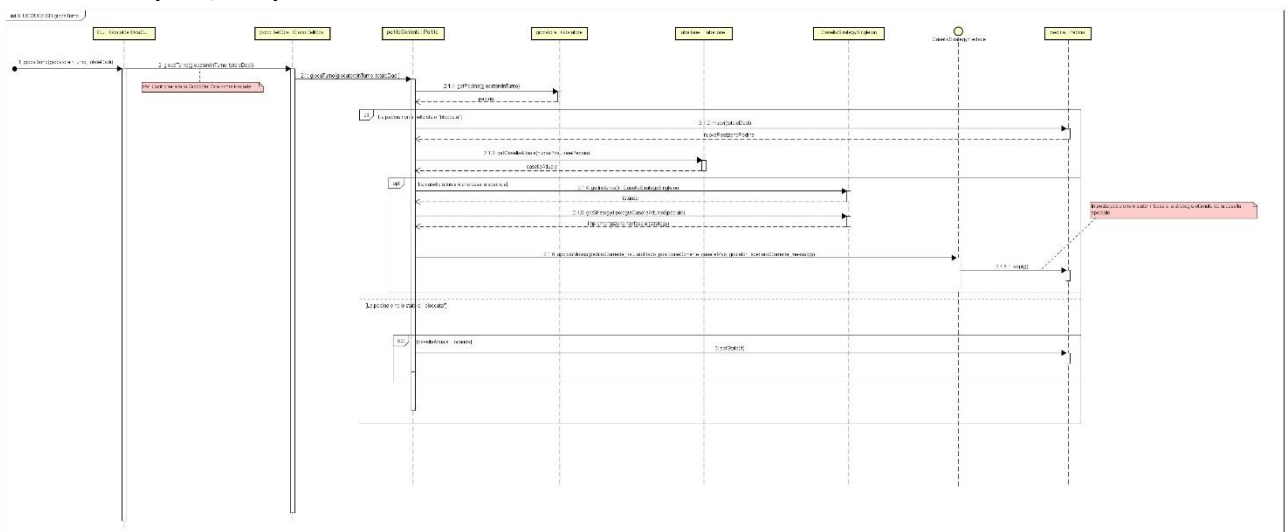
- Set pedina (Giocatore 1) (UC1/UC3)



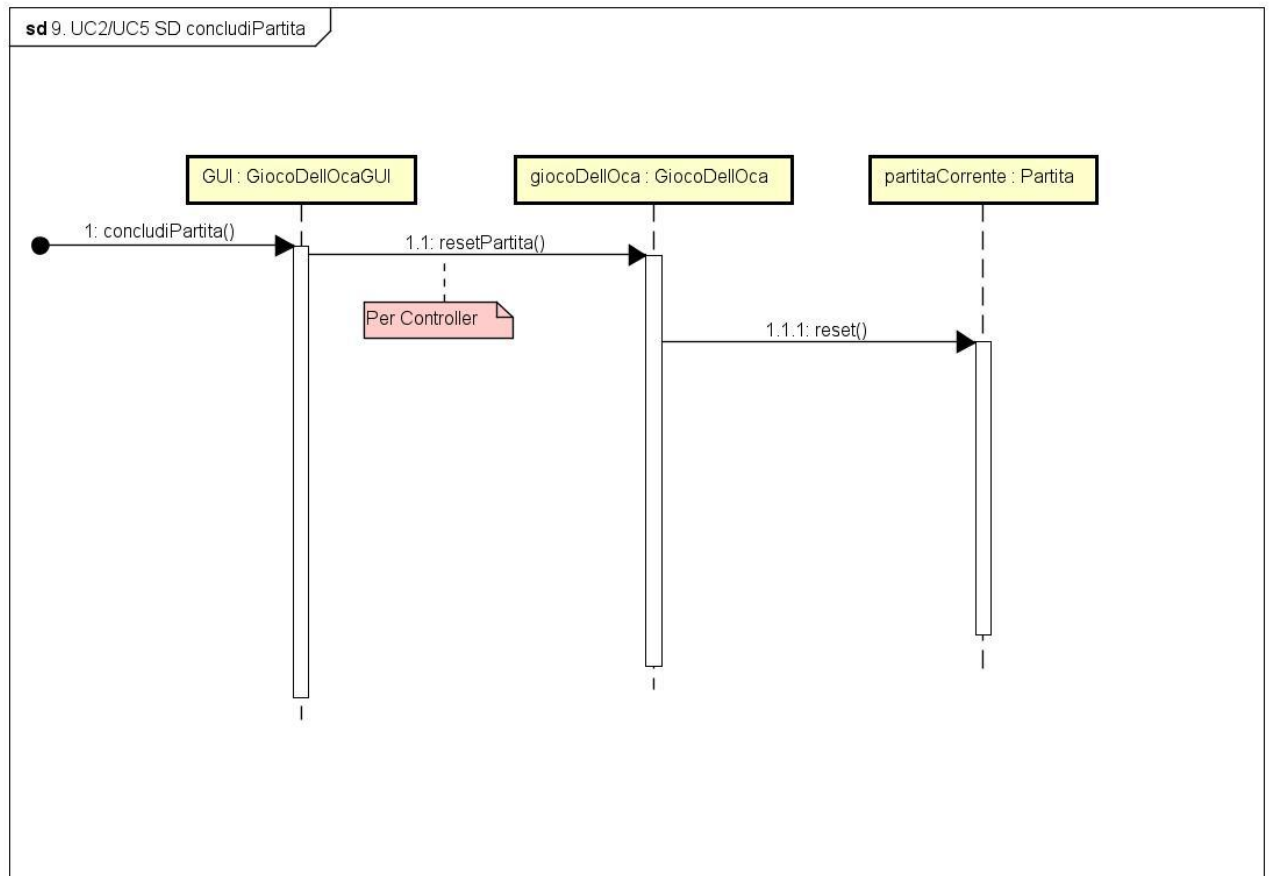
- Avvia partita (UC2/UC5)



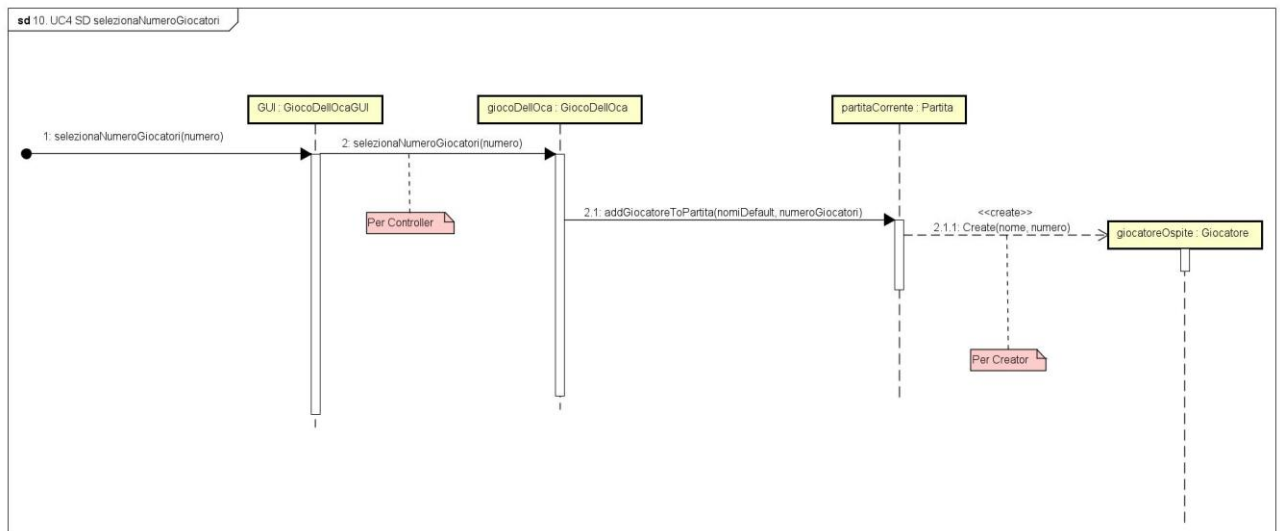
- Gioca turno (UC2/UC5)



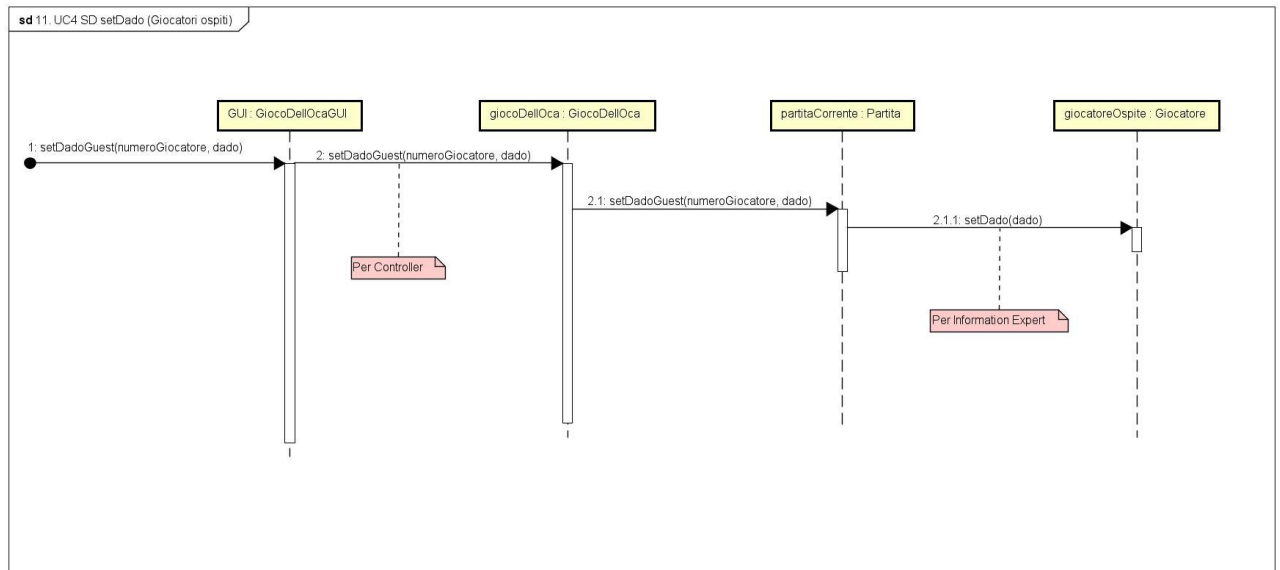
- **Concludi partita (UC2/UC5)**



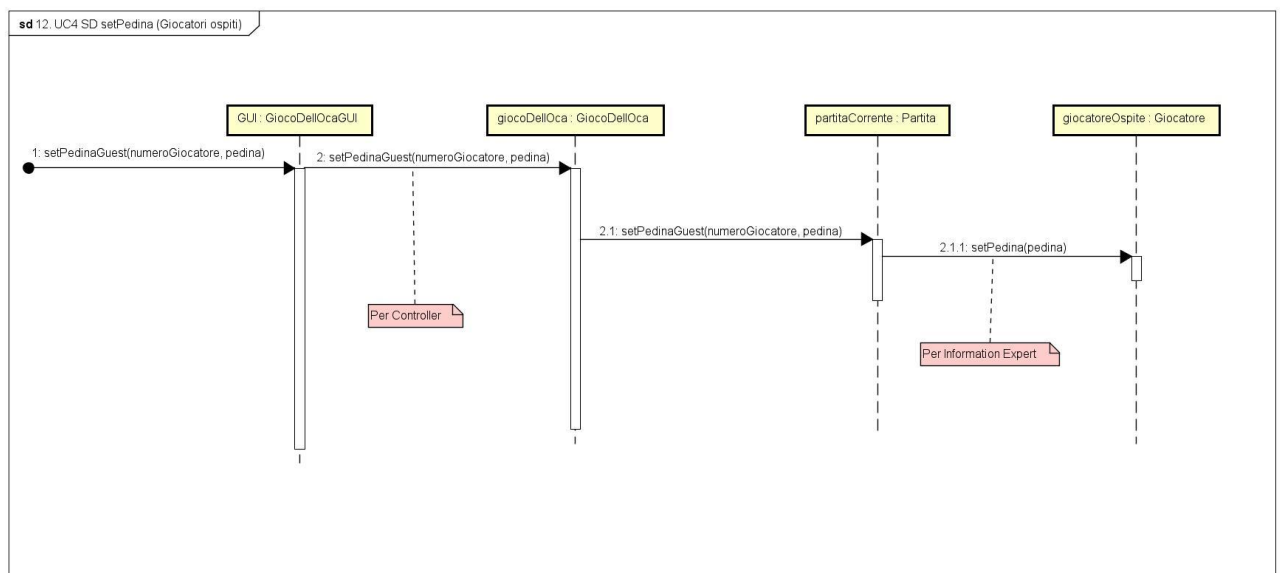
- **Seleziona numero giocatori (UC4)**



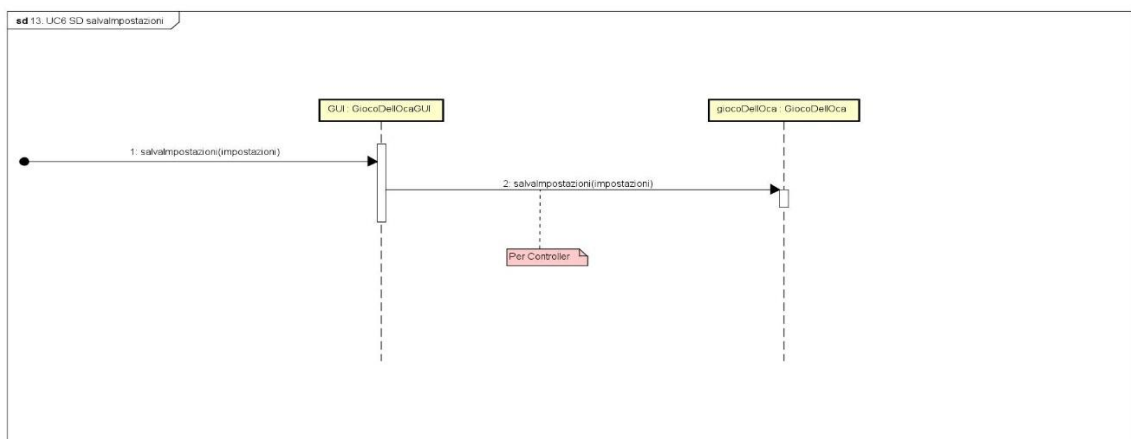
- Set dado (Giocatori ospiti) (UC4)



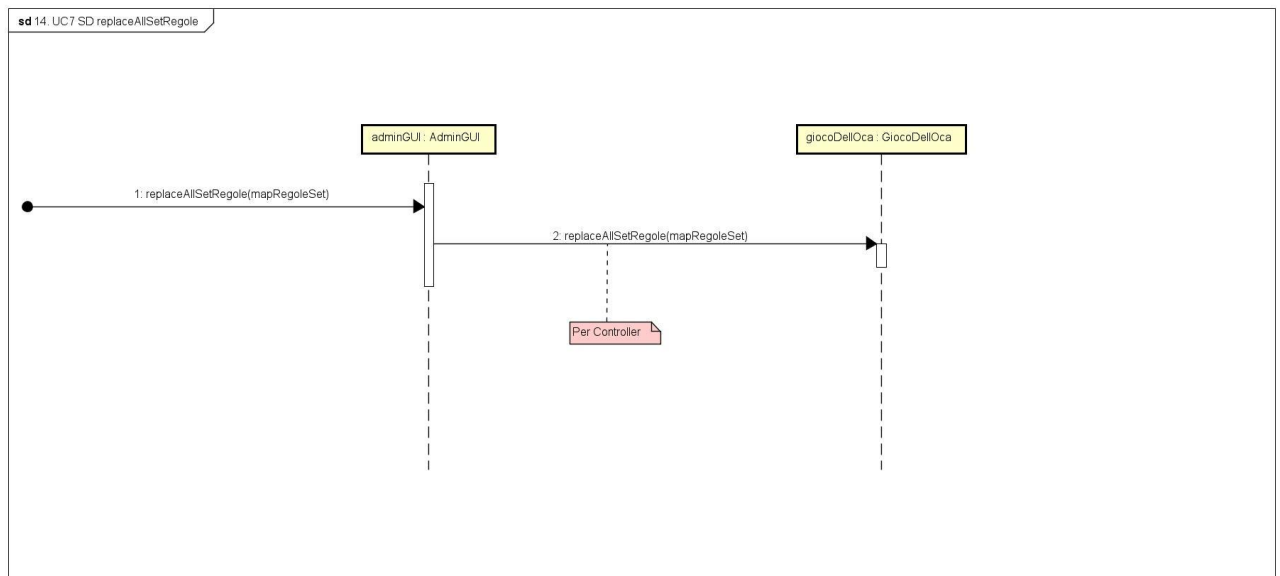
- Set pedina (Giocatori ospiti) (UC4)



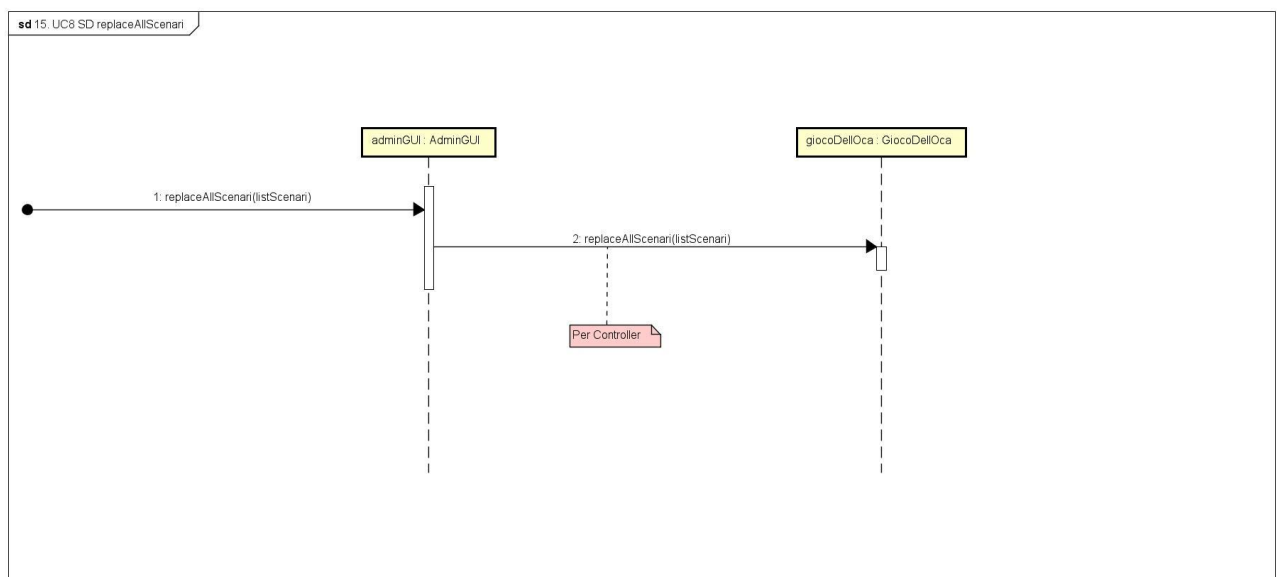
- Salva impostazioni (UC6)



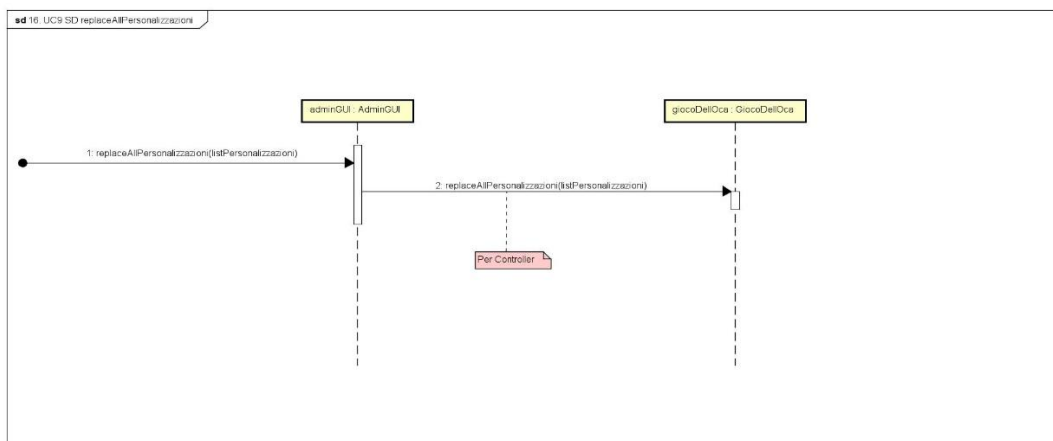
- Replace all set regole (UC7)



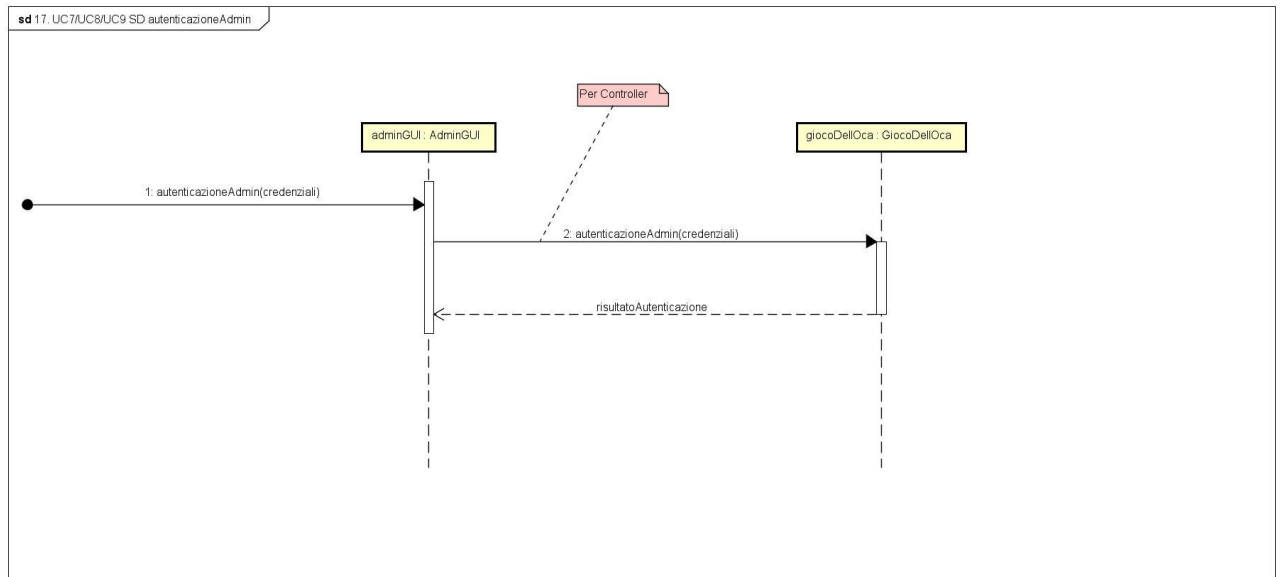
- Replace all scenari (UC8)



- Replace all personalizzazioni (UC9)



- Autenticazione admin (UC7/UC8/UC9)



TESTING

Il testing rappresenta una fase fondamentale del processo di sviluppo del software, poiché consente di garantirne robustezza ed efficienza, riducendo in modo significativo i costi di manutenzione. Sebbene la probabilità di individuare malfunzionamenti aumenti con il numero di test eseguiti, risulta impraticabile testare completamente un programma: le combinazioni di input possibili sono infatti estremamente numerose e non sempre riproducibili in tempi ragionevoli. Tuttavia, un'attività di testing accuratamente pianificata permette di individuare comportamenti anomali o indesiderati, contribuendo a rendere il software conforme alle specifiche e pienamente funzionante.

Le tipologie principali di test sono due:

- **Test Unitari (Unit Test)** – Si tratta di test mirati a verificare la correttezza delle più piccole parti del codice. In Java, lo scopo dei test unitari è controllare il comportamento dei singoli metodi confrontando i valori attesi con quelli effettivi. Tra i vari strumenti disponibili per questo tipo di test, il framework JUnit è attualmente il più diffuso per automatizzare il testing delle unità software in applicazioni Java.
- **Test Funzionali** – Sono test che verificano il corretto funzionamento dell'intero sistema, includendo anche l'interfaccia grafica. Considerano il software come una “scatola nera”, fornendo input e controllando la correttezza degli output prodotti.

Per il testing dell'applicazione sviluppata si è scelto di adottare principalmente test unitari, implementati tramite il framework JUnit e seguendo un approccio *bottom-up*, collaudando quindi gradualmente le singole componenti che compongono il sistema completo.

I test sono stati eseguiti sulla seguente classe:

- GiocoDellOcaTest

I metodi testati sono i seguenti metodi provenienti dalla classe GiocoDellOca:

1. `void impostazioniTest()`
2. `void impostaRegoleSingoleTest()`
3. `void impostaRegoleDaSetTest()`
4. `void impostaScenarioTest()`
5. `void impostaPedinaTest()`
6. `void impostaDadoTest()`
7. `void eseguiTurnoTest()`
8. `void aggiungiGiocatoriOspitiTest()`
9. `void configuraGiocatoreCorrenteTest()`
10. `void autenticazioneAdminTest()`
11. `void replaceAllScenariTest()`
12. `void replaceAllRegoleTest()`
13. `void replaceAllPersonalizzazioniTest()`

void impostazioniTest()

Test del salvataggio delle impostazioni per avviare il gioco.

Condizione iniziale: che esista una variabile `giocoDellOca`.

Condizione da testare: che esistano i giocatori, le personalizzazioni, lo scenario e le regole impostate.

void impostaRegoleSingoleTest()

Test dell'impostazione di una regola singola.

Condizione iniziale: che esista una variabile `giocoDellOca` e che si stia configurando una nuova partita.

Condizione da testare: perché il metodo funzioni si deve verificare che il numero di regole che vengono salvate nelle impostazioni sia 0 se la tabella è vuota (non sono state aggiunte manualmente via GUI) oppure uguale al numero di record passato dalla tabella se quest'ultima è stata valorizzata.

void impostaRegoleDaSetTest()

Test dell'impostazione di un set di regole.

Condizione iniziale: che esista una variabile `giocoDellOca` e che si stia configurando una nuova partita.

Condizione da testare: perché il metodo funzioni si deve verificare che il numero di regole che vengono salvate nelle impostazioni sia 0 se la tabella è vuota (non sono state aggiunte manualmente via GUI) oppure uguale al numero di record passato dalla tabella se quest'ultima è stata valorizzata.

void impostaScenarioTest()

Test dell'impostazione di uno scenario.

Condizione iniziale: che esista una variabile `giocoDellOca` e che si stia configurando una nuova partita.

Condizione da testare: perché il metodo funzioni si deve verificare che lo scenario scelto esista tra quelli definiti nella lista di `giocoDellOca` altrimenti viene impostato lo scenario di default.

void impostaPedinaTest()

Test dell'impostazione di una pedina abbinata ad un giocatore.

Condizione iniziale: che esista una variabile `giocoDellOca`, che si stia configurando una nuova Partita e che esista un giocatore per poter associare la pedina.

Condizione da testare: perché il metodo funzioni si deve verificare che la pedina scelta esista e sia stata associata correttamente al giocatore.

void impostaDadoTest()

Test dell'impostazione di un dado abbinato ad un giocatore.

Condizione iniziale: che esista una variabile `giocoDelloOca`, che si stia configurando una nuova Partita e che esista un giocatore per poter associare il dado.

Condizione da testare: perché il metodo funzioni si deve verificare che il dado scelto esista e sia stata associato correttamente al giocatore.

void eseguiTurnoTest()

Test dell'esecuzione di un turno di un giocatore.

Condizione iniziale: che esista una variabile `giocoDelloOca`, che sia stata configurata e avviata una partita con delle impostazioni di default, e che esista una pedina.

Condizione da testare: perché il metodo funzioni si deve verificare che per ogni possibile tipologia di casella sulla quale la pedina può atterrare, la posizione e lo stato di quest'ultima siano corrette.

void aggiungiGiocatoriOspitiTest()

Test dell'aggiunta giocatori ospiti.

Condizione iniziale: che esista una variabile `giocoDelloOca` e che si stia configurando una partita.

Condizione da testare: perché il metodo funzioni si deve verificare che il numero di giocatori aggiunti corrisponda effettivamente al numero di giocatori registrato per la partita corrente.

void configuraGiocatoreCorrenteTest()

Test della configurazione giocatore corrente da fare muovere lungo il tabellone.

Condizione iniziale: che esista una variabile `giocoDelloOca` e che stia configurando/sia già stata configurata una nuova partita.

Condizione da testare: perché il metodo funzioni si deve verificare che, qualora le tabelle preposte alla definizione di pedina e dado siano vuote, vengano associate delle personalizzazioni di default, e il giocatore viene impostato come giocatore corrente.

void autenticazioneAdminTest()

Test della autenticazione come admin per poter definire le operazioni

Condizione iniziale: che esista una variabile `giocoDelloOca` e che si voglia accedere al pannello amministratore.

Condizione da testare: perché il metodo funzioni si deve passare in input la coppia corretta username e password "admin" "admin", altrimenti viene restituito un booleano che blocca la funzionalità.

void replaceAllScenariTest()

Test rimpiazzo scenari di default con scenari nuovi.

Condizione iniziale: che esista una variabile `giocoDelloca` e che gli scenari vecchi esistano già per poter fare il confronto.

Condizione da testare: perché il metodo funzioni si deve verificare che i nuovi elementi siano effettivamente restituiti dalla `getListaScenari` di `giocoDelloca` e siano diversi dai vecchi.

void replaceAllRegoleTest()

Test rimpiazzo set regole di default con regole nuovi.

Condizione iniziale: che esista una variabile `giocoDelloca` e che i set di regole vecchi esistano già per poter fare il confronto.

Condizione da testare: perché il metodo funzioni si deve verificare che i nuovi elementi siano effettivamente restituiti dalla `getMapRegoleSet` di `giocoDelloca` e siano diversi dai vecchi.

void replaceAllPersonalizzazioniTest()

Test rimpiazzo personalizzazioni di default con personalizzazioni nuove.

Condizione iniziale: che esista una variabile `giocoDelloca` e che le personalizzazioni vecchie esistano già per poter fare il confronto.

Condizione da testare: perché il metodo funzioni si deve verificare che i nuovi elementi siano effettivamente restituiti dalla `getListaPersonalizzazioni` di `giocoDelloca` e siano diversi dai vecchi.

A seguito dell'esecuzione dei test unitari tramite JUnit, si è provveduto, quando necessario, a correggere le parti di codice responsabili del fallimento dei test, così da ottenere un software più corretto, stabile e robusto, e lo stesso è stato fatto per i test funzionali.

REFACTORING

Durante lo sviluppo iterativo del software, sia il codice sia la documentazione sono stati oggetto di un processo continuo di refactoring, finalizzato a migliorarne la correttezza, la manutenibilità e la qualità complessiva. Nelle prime iterazioni l'obiettivo principale è stato ottenere un sistema funzionante; successivamente, il codice è stato riorganizzato all'interno delle classi più appropriate, in modo da garantire un'assegnazione delle responsabilità coerente con quanto definito in fase di progettazione. Inoltre, per incrementare la pulizia e la leggibilità del codice, durante le diverse iterazioni sono state introdotte funzioni e classi di *Utilities* che hanno contribuito a rendere l'implementazione più modulare ed efficiente.

TEST DI ACCETTAZIONE

Al termine delle attività necessarie a completare le fasi precedenti e dopo un'accurata revisione dell'intero codice sorgente, è stato eseguito il test di accettazione finale. Sono state effettuate diverse simulazioni sia dal punto di vista dei Giocatori sia da quello dell'Amministratore, verificando tutte le principali funzionalità del sistema per assicurarsi che il programma operasse correttamente e che i casi d'uso implementati venissero portati a termine con successo.

DESIGN PATTERN APPLICATI

Durante il processo di sviluppo, a seguito di una revisione del codice, è emersa l'opportunità di introdurre alcuni design pattern appartenenti ai Gang of Four (GoF), con l'obiettivo di migliorare l'efficienza e la qualità strutturale del software. In particolare, sono stati adottati i seguenti pattern:

- **Pattern Singleton**

Singleton è un *creational pattern* che garantisce l'esistenza di una sola istanza di una determinata classe, fornendo un punto di accesso globale tramite un metodo statico, solitamente denominato *getInstance()*.

Nel nostro contesto, la classe **CasellaStrategySingleton** è stata resa **Singleton** per impedire la creazione di più istanze e assicurare una gestione centralizzata e univoca delle strategie associate alle caselle speciali.

- **Pattern Strategy**

Strategy è un *behavioral pattern* che permette di definire una famiglia di algoritmi intercambiabili, incapsulandoli in classi separate e rendendoli sostituibili a *runtime* senza modificare il codice che li utilizza.

Nel nostro caso, il pattern è stato utilizzato per gestire in modo flessibile i comportamenti differenti delle varie **caselle speciali** del tabellone. Questo ha permesso di estendere o modificare le logiche delle caselle senza alterare la struttura principale del programma.

Per implementare il pattern **Strategy**, sono state introdotte le seguenti componenti:

- **Interfaccia CasellaStrategyInterface**
- **Classe OcaStrategy**
- **Classe LocandaStrategy**
- **Classe ScheletroStrategy**
- **Classe PonteStrategy**
- **Classe PrigioneStrategy**
- **Classe LabirintoStrategy**

